

STUDY GUIDE · FROM YOUR FIRST CONTAINER TO A FULL APPLICATION

Docker From Zero

A Complete Hands-On Learning Lab

23 lessons for complete beginners: images, containers, the CLI, ports, environment variables, Dockerfiles, layers and cache, .dockerignore, volumes, bind mounts, networking, Compose, logs, inspect, resources, security, optimization, Docker Hub, troubleshooting and the capstone, plus a glossary and 25 interview questions.

Sufyan Ahmad · DevOps Engineer

<https://github.com/sufyanahmadkamboh/sufyan-devops-docker-from-zero>

Contents

[Study guide](#)

[Docker Introduction](#)

[Docker Installation](#)

[Images](#)

[Containers](#)

[Docker CLI](#)

[Container Lifecycle](#)

[Ports](#)

[Environment Variables](#)

[Dockerfile](#)

[Image Layers](#)

[.dockerignore](#)

[Volumes](#)

[Bind Mounts](#)

[Docker Networking](#)

[Docker Compose](#)

[Logs and Debugging](#)

[Docker Inspect](#)

[Resource Management](#)

[Security Basics](#)

[Image Optimization](#)

[Docker Hub and Registries](#)

[Troubleshooting](#)

[Capstone: Everything Together](#)

[Glossary](#)

[Interview questions](#)

[Knowledge checklist](#)

Each lesson: what it is, why we need it, how it works, a hands-on lab, an experiment, a deliberate failure and its investigation, common mistakes, best practices, a challenge with its solution, and where you use it at work.

The hands-on labs, the guided tutorial and the troubleshooting scenarios are in the repository.

Study guide

Everything you need to study Docker from zero, also as one printable PDF: [study-guide.pdf](#) (the 23 lessons, the glossary, 25 interview questions and the knowledge checklist).

How to study with this repository

Step	Do this	Time
1	Read the lesson in docs/ to understand the idea	15 min per lesson
2	Follow the matching chapter of the guided tutorial/ , typing every command	30-60 min per chapter
3	Do the lab in labs/ on your own, including "Break it"	30-45 min per lab
4	Solve the challenges without opening the solutions	15-30 min per topic
5	Practise the troubleshooting scenarios	20 min each
6	Build and verify the capstone	60 min
7	Tick the knowledge checklist and answer the interview questions	30 min

Total: about 15-20 hours for a careful first pass. The [roadmap](#) maps every stage to its tutorial chapter, lab and lesson.

Prerequisites

Basic terminal skills (`cd` , `ls` , `cat` , editing a text file), Docker installed ([docs/02](#)), and Git. No programming knowledge is needed: the example applications are small and you never have to change their logic.

Also in this folder

- [glossary.md](#): every Docker term used in the lab, in plain language
- [interview-questions.md](#): 25 questions with answers that point back to the labs
- [tools/build_pdf.py](#): rebuilds the PDF from the Markdown lessons (`pip install markdown`)

Docker Introduction

Lesson 01 · about 20 minutes · you need: nothing yet (Docker installed is a bonus; see [02 · Installation](#))

What is it?

Docker is a tool that packages an application **together with everything it needs to run** (the right language version, libraries, configuration files, a small operating-system userland) into one unit called an **image**. When you start an image, you get a **container**: a normal process on your computer that is isolated from the other processes, with its own files, its own network address and its own view of the system.

Think of a shipping container. The port crane does not care whether the container holds bananas or car parts: every container has the same shape, so every crane, ship and truck can move it. Docker does the same for software. Every container is started, stopped, moved and inspected with the same commands, whatever is inside.

Without Docker

"It works on my machine"

your laptop: Python 3.12
test server: Python 3.10
production: Python 3.14 ???

With Docker

The machine is part of the package

image: python:3.14-slim
+ Flask 3.1.3
+ your app.py
same image on every computer

Why do we need it?

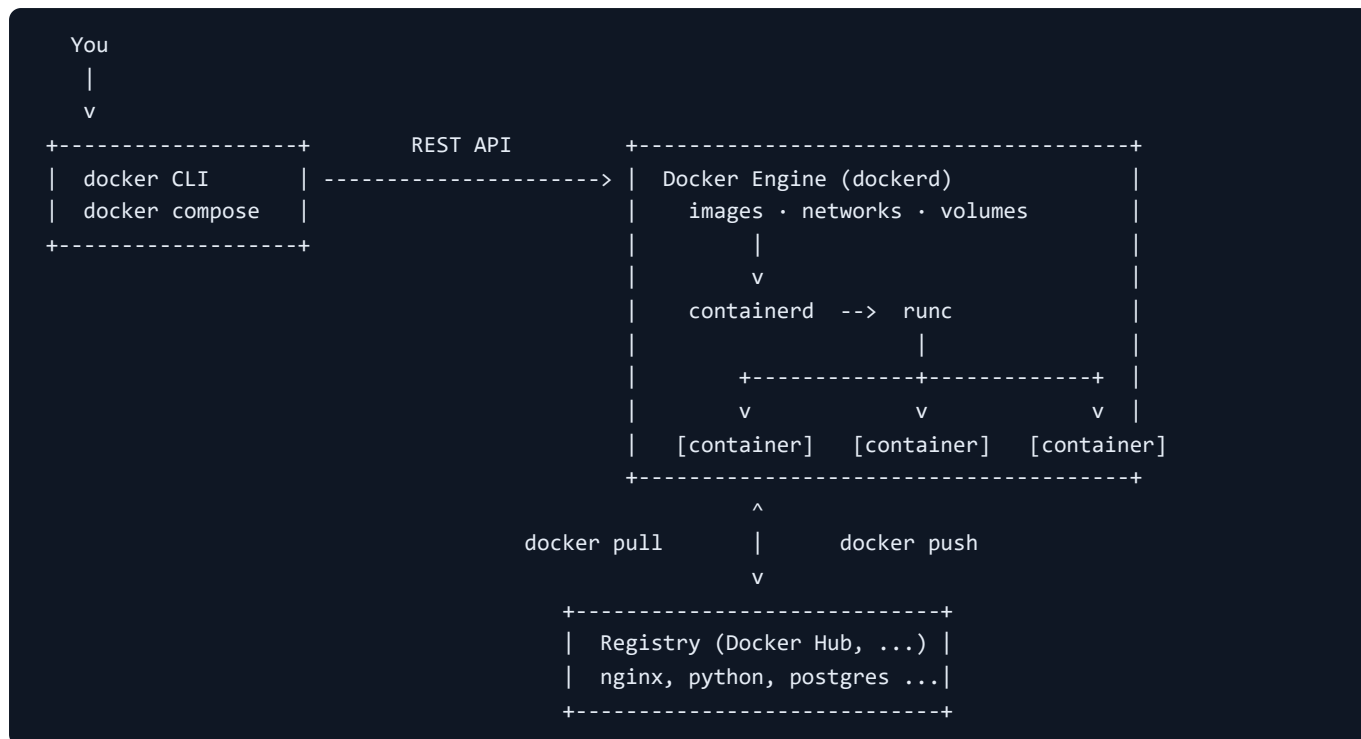
Before containers, installing an application meant following a long list of manual steps on every server: install this package, that library, copy these files, set those variables. Every server ended up slightly different, and the differences caused bugs that nobody could reproduce.

Docker solves four everyday problems:

Problem	How Docker helps
"It works on my machine"	The image contains the exact versions, so it runs the same everywhere
Slow, manual setup	<code>docker run</code> starts a complete PostgreSQL, nginx or Python in seconds
Conflicts between apps	Two containers can use different Python versions on the same computer
Messy clean-up	Remove the container and nothing is left behind on your computer

How does it work?

Docker is a **client-server** system. The command you type (`docker run ...`) is only the client. It sends a request to the **Docker Engine** (the daemon, `dockerd`), which does the work: it downloads images from a **registry**, and asks **containerd** and **runc** to create the isolated process.



The isolation comes from features of the **Linux kernel**:

- **namespaces** give each container its own view: its own process list (PID 1 is your app), its own hostname, its own network interfaces, its own filesystem tree;
- **cgroups** limit how much CPU and memory a container may use;
- a **layered filesystem** lets many containers share the same read-only image files.

A container is **not** a virtual machine. A VM runs a complete second operating system with its own kernel. A container shares the kernel of the host and only adds isolation around a process. That is why containers start in milliseconds and use little memory.



On Windows and macOS, Docker Desktop runs a small Linux virtual machine in the background, and your containers run inside it. You will not notice it in daily use.

Three words you will use in every lesson:

Word	Meaning	Analogy
Image	A read-only package: files + settings + the command to run	A recipe, or a class in programming
Container	A running (or stopped) instance of an image	The cooked meal, or an object

Word	Meaning	Analogy
Registry	A server that stores images (Docker Hub is the public one)	An app store for images

Prerequisites

- Basic terminal skills: `cd`, `ls`, `cat`, running a command and reading its output.
- For the hands-on parts: Docker installed ([02 · Installation](#)).

Hands-on Lab

Check that the Docker client can talk to the Docker Engine. `docker version` prints two parts: **Client** (the CLI you typed) and **Server** (the engine). If the Server part is missing, the engine is not running.

```
docker version
```

```
Client:
Version:      29.4.3
API version:  1.54
Go version:   go1.26.2
Git commit:   055a478
Built:        Wed May  6 17:10:36 2026
OS/Arch:      windows/amd64
Context:      desktop-linux

Server: Docker Desktop 4.74.0 (227015)
Engine:
Version:      29.4.3
...
```

Then run your very first container:

```
docker run hello-world
```

The full guided version of this is [labs/01-first-container](#).

Expected Result

- `docker version` shows a **Client** section and a **Server** section (Docker Engine).
- `docker run hello-world` prints a message that starts with `Hello from Docker!` and then explains the four steps Docker just performed (contact the daemon, pull the image, create the container, stream the output to your terminal).

Experiment

Run `hello-world` a second time. Compare the output with the first time: the lines about "Unable to find image locally" and "Pulling from library/hello-world" are gone. The image is now stored on your computer, so Docker reuses it.

```
docker run hello-world
```

Now list the containers, including stopped ones. Every `docker run` created a **new** container:

```
docker ps -a
```

Break It

Ask Docker for an image that does not exist:

```
docker run hello-wrld
```

Troubleshoot It

- **Observe:** Docker first says `Unable to find image 'hello-wrld:latest' locally`, then fails with `pull access denied ... repository does not exist or may require 'docker login'` (the exact wording depends on your Docker version).
- **Investigate:** read the image name in the error. Docker added `:latest` (and, behind the scenes, `docker.io/library/`) to it: that is how Docker expands short image names. Docker Hub answered "no such repository". The `docker login` hint is a red herring: logging in will not make a misspelled image exist.
- **Root cause:** a typo in the image name (`hello-wrld`).
- **Fix:** use the correct name `hello-world`.
- **Verify:** `docker run hello-world` prints `Hello from Docker!`.

If instead you see `Cannot connect to the Docker daemon` or `error during connect`, the client works but the engine is not running: start Docker Desktop (Windows/macOS) or run `sudo systemctl start docker` (Linux). See [02 · Installation](#).

Common Mistakes

- Thinking a container is a small virtual machine. It is an isolated process on a shared kernel.
- Thinking `docker` is the engine. The CLI is only a client; the work happens in `dockerd`.
- Expecting Windows programs to run in a Linux container. Containers share the host's kernel type; this course uses Linux containers only.
- Ignoring the output. Docker's messages are usually precise: read them before searching online.

Best Practices

- Learn the three nouns (image, container, registry) first; every other topic builds on them.
- Use official or verified images as a starting point (`nginx`, `python`, `postgres`).
- Always say which **version** of an image you want (`nginx:1.30-alpine`, not just `nginx`); lesson [03 · Images](#) explains why.

Challenge

Without looking at the solution: find out **which version** of the Docker Engine (the server) you are running, using only one command and its output.

Solution

▼ Show the solution

`docker version` shows the Server version in its second half. A shorter way prints only that value with a Go template (you will learn templates in [17 · docker inspect](#)):

```
docker version --format '{{.Server.Version}}'
```

```
29.4.3
```

Verification

You understood this lesson if you can explain, in your own words:

- [] the difference between an image and a container;
- [] what the Docker client does and what the Docker Engine does;
- [] why a container starts faster than a virtual machine;
- [] what a registry is and where `hello-world` came from.

Real-World Usage

As a DevOps engineer you use Docker every day:

- **Packaging:** developers hand over an image instead of installation instructions.
- **Local environments:** a new team member starts the whole stack with one command.
- **Testing:** CI systems run tests inside the same image that goes to production.
- **Running in production:** container platforms (later in your career: Kubernetes, managed container services) all run the same OCI images you learn to build here.

Key Takeaways

- An **image** is a package; a **container** is a running instance of it.
- The `docker` command is a client; the **Docker Engine** does the work.
- Containers are isolated **processes** that share the host kernel; they are not VMs.
- Images come from a **registry**; Docker Hub is the default one.
- Next: [02 · Docker Installation](#).

Docker Installation

Lesson 02 · 20–45 minutes (mostly download time) · you need: a computer with admin rights, about 10 GB of free disk space

What is it?

Installing Docker gives you two things:

- 1. the **Docker Engine** (`dockerd`), the background service that builds images and runs containers;
- 2. the **Docker CLI** (`docker`) and the **Compose plugin** (`docker compose`), the commands you type.

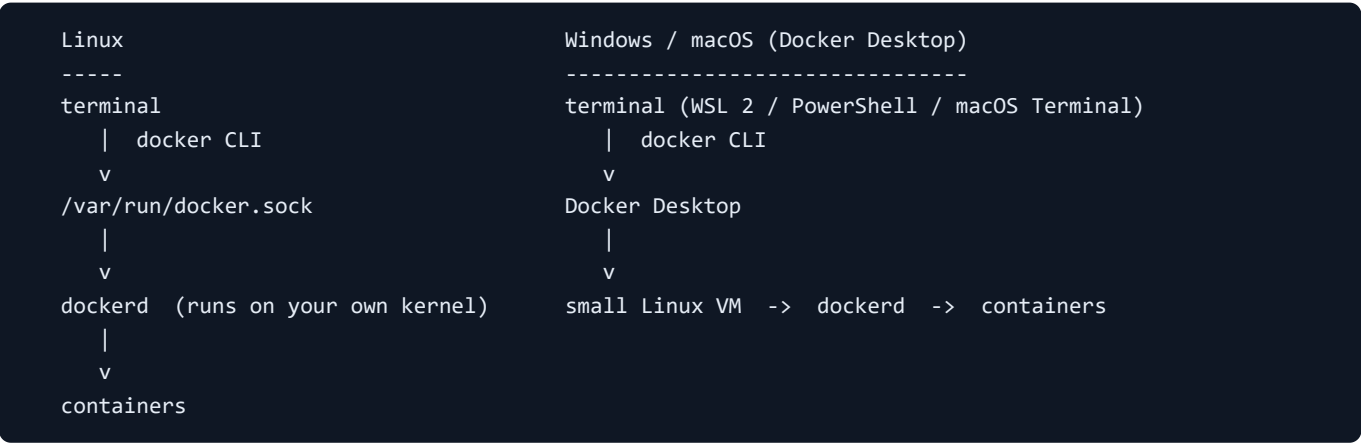
There are two ways to get them:

Your computer	What to install	What you get
Windows 10/11	Docker Desktop with the WSL 2 backend	Engine in a small Linux VM, CLI on Windows and in WSL
macOS (Intel or Apple Silicon)	Docker Desktop for Mac (pick the matching chip)	Engine in a small Linux VM, CLI in Terminal
Linux (Ubuntu, Debian, ...)	Docker Engine from Docker's official apt repository	Engine runs directly on your kernel, no VM

Why do we need it?

Every lab in this repository talks to a real Docker Engine. A correct installation avoids the most frustrating beginner problems: "permission denied", "cannot connect to the Docker daemon", paths that do not work in your terminal, and very old versions from the operating system's own package list (which miss features this course uses, such as `docker compose`).

How does it work?



The CLI talks to the engine through a **socket** (a special file, `/var/run/docker.sock` on Linux). Whoever can write to that socket can tell Docker to do anything, including mounting the whole host filesystem into a container. That is why access to Docker is a powerful permission.

Prerequisites

- **Windows:** Windows 10 22H2 or Windows 11, 64-bit, virtualization enabled in the BIOS/UEFI (Task Manager → Performance → CPU shows "Virtualization: Enabled").
- **macOS:** a currently supported macOS version; know whether you have an Intel or Apple chip (→ About This Mac).
- **Linux:** a 64-bit Ubuntu (22.04, 24.04, 26.04) or Debian (12, 13) with `sudo` rights.
- **Git**, to download this repository.

Hands-on Lab

Option A · Windows: Docker Desktop + WSL 2 (recommended setup)

1. Open PowerShell **as Administrator** and install WSL 2 with Ubuntu (restart when asked):

```
wsl --install -d Ubuntu
```

2. Download Docker Desktop from <https://docs.docker.com/desktop/setup/install/windows-install/>, run the installer, keep **"Use WSL 2 instead of Hyper-V"** selected.
3. Start Docker Desktop, open **Settings** → **Resources** → **WSL integration** and switch on your Ubuntu distribution.
4. Open the **Ubuntu** app (this is your WSL terminal). Use it for all labs in this course.

Option B · macOS: Docker Desktop

1. Download Docker Desktop from <https://docs.docker.com/desktop/setup/install/mac-install/> (choose **Apple Silicon** or **Intel chip**).
2. Drag Docker to Applications, start it, wait until the whale icon in the menu bar says "Docker Desktop is running".
3. Use the normal **Terminal** app for the labs.

Option C · Linux (Ubuntu): Docker Engine from Docker's apt repository

These commands follow the official guide at <https://docs.docker.com/engine/install/ubuntu/>. They change your system, so read each one before running it.

1. Remove old, unofficial packages if they exist (it is fine if apt says none are installed):

```
for pkg in docker.io docker-doc docker-compose podman-docker containerd runc; do sudo apt-get remove -y $pkg; done
```

2. Add Docker's signing key and repository:

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo
tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt-get update
```

3. Install the engine, the CLI, Buildx and the Compose plugin:

```
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

On **Debian**, replace `ubuntu` with `debian` in both URLs.

Alternative: the convenience script. Docker also publishes a script that does all of the above in one go. It is fine for a personal learning VM, but it runs as root and you do not see what it changes, so do not use it on servers you share with others:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

4. **Run Docker without `sudo`** (optional, but every lab here assumes it). Add yourself to the `docker` group, then log out and back in:

```
sudo usermod -aG docker "$USER"
```

Security note: members of the `docker` group can start a container that mounts `/` from the host and changes any file as root. Being in the `docker` group is effectively the same as having root on that machine. That is acceptable on your own learning machine. On shared servers, only trusted administrators should be in this group.

Everyone: get Git and this repository

Check Git (install it from <https://git-scm.com/downloads> or `sudo apt-get install -y git` if the command is missing):

```
git --version
```

Clone the course and go into it (in WSL on Windows):

```
git clone https://github.com/sufyanahmadkamboh/sufyan-devops-docker-from-zero.git
cd sufyan-devops-docker-from-zero
```

Everyone: verify the installation

The client and the server should both answer:

```
docker version
```

```
Client:
Version:      29.4.3
API version:  1.54
Go version:   go1.26.2
Git commit:   055a478
Built:        Wed May 6 17:10:36 2026
OS/Arch:      windows/amd64
Context:      desktop-linux

Server: Docker Desktop 4.74.0 (227015)
Engine:
Version:      29.4.3
API version:  1.54 (minimum version 1.40)
Go version:   go1.26.2
...
```

Compose is a plugin of the CLI (two words, `docker compose`, not the old `docker-compose`):

```
docker compose version
```

```
Docker Compose version v5.1.4
```

And the classic smoke test:

```
docker run hello-world
```

Which terminal should I use?

System	Recommended terminal	Folder of the current directory in commands
Windows	WSL 2 (Ubuntu app)	<code>"\$(pwd)"</code>
Windows	PowerShell (works for most labs)	<code>\${PWD}</code> instead of <code>"\$(pwd)"</code>
Windows	Git Bash (works, with one setting)	<code>"\$(pwd)"</code> , after <code>export MSYS_NO_PATHCONV=1</code>
macOS / Linux	the normal Terminal	<code>"\$(pwd)"</code>

Why the Git Bash setting? Git Bash "helpfully" converts anything that looks like a Unix path into a Windows path. `docker exec web ls /usr/share/nginx/html` silently becomes `ls C:/Program Files/Git/usr/share/nginx/html`, which does not exist inside the container. `export MSYS_NO_PATHCONV=1` switches that conversion off for the current terminal.

Expected Result

- `git --version` prints `git version 2.x`.
- `docker version` shows **Client** and **Server** sections without errors.
- `docker compose version` prints `Docker Compose version v2.x` or newer.
- `docker run hello-world` prints `Hello from Docker!`.

Experiment

`docker info` shows how your engine is set up. Look for `Server Version`, `Operating System` (on Docker Desktop it says "Docker Desktop"), `CPUs`, `Total Memory` and `Docker Root Dir` (where images and volumes are stored):

It prints a long report. For now, ask only for the fields that matter here, with `--format` :

```
docker info --format 'Server Version: {{.ServerVersion}}
Operating System: {{.OperatingSystem}}
CPUs: {{.NCPU}}
Total Memory: {{.MemTotal}} bytes
Docker Root Dir: {{.DockerRootDir}}'
```

```
Server Version: 29.4.3
Operating System: Docker Desktop
CPUs: 14
Total Memory: 16486322176 bytes
Docker Root Dir: /var/lib/docker
```

Run plain `docker info` too and scroll through it: the same fields are in there, together with the plugins your installation includes, the storage driver and the number of containers and images.

Break It

On Linux, stop the engine and try a command (skip this on Docker Desktop: there you would quit Docker Desktop from its menu instead):

```
sudo systemctl stop docker.socket
docker ps
```

Troubleshoot It

Message	What it means	Fix
Cannot connect to the Docker daemon at unix:///var/run/docker.sock. Is the docker daemon running?	The CLI works, the engine is not running	Linux: <code>sudo systemctl start docker</code> · Desktop: start Docker Desktop and wait for "running"
permission denied while trying to connect to the Docker daemon socket	You are not in the <code>docker</code> group (or did not log out/in after adding yourself)	<code>sudo usermod -aG docker "\$USER"</code> , then log out and back in, then <code>docker ps</code>
docker: command not found in WSL	WSL integration is off for this distribution	Docker Desktop → Settings → Resources → WSL integration → switch it on
docker-compose: command not found	You typed the old v1 command	Use <code>docker compose</code> (with a space)
ls: cannot access 'C:/Program Files/Git/...'	Git Bash path conversion	<code>export MSYS_NO_PATHCONV=1</code> , or use WSL

Restart after the Break It step:

```
sudo systemctl start docker
docker ps
```

Common Mistakes

- Installing `docker.io` from Ubuntu's own repository and then following a guide for Docker's official packages. Pick one source; this course assumes Docker's official repository.
- Forgetting to log out and back in after `usermod -aG docker`: group changes only apply to new logins.
- Running every command with `sudo` "to be safe": files created by root inside bind mounts then become hard to edit, and you get used to giving everything root.
- On Windows, cloning the repository into `C:\...` and working in WSL: it works, but file access across the boundary is slow. Clone it inside WSL (`~/`) instead.

Best Practices

- Keep Docker Desktop / Docker Engine updated; security fixes ship regularly.
- Give Docker Desktop enough resources (Settings → Resources): 2 CPUs and 4 GB RAM are plenty for this course.
- Treat the `docker` group like `sudo`: only for people who should have root.
- Use WSL 2 on Windows: the labs use Linux commands, and they behave exactly as written.

Challenge

Find out, with commands only, **where your engine stores its data** and **how many CPUs** it can use.

Solution

▼ Show the solution

Both values are in `docker info`. A Go template prints just those fields:

```
docker info --format 'Docker Root Dir: {{.DockerRootDir}} · CPUs: {{.NCPU}}'
```

```
Docker Root Dir: /var/lib/docker · CPUs: 14
```

On Docker Desktop the root directory is inside the Linux VM, not a folder you can open in Windows Explorer or Finder.

Verification

- `[] docker version` shows a Server section.
- `[] docker compose version` works.
- `[] docker run hello-world` works **without** `sudo`.
- `[]` You cloned this repository and you are inside its folder.

- [] You know which terminal you will use for the labs.

Real-World Usage

- Every developer workstation and CI runner in a container-based team has Docker (or a compatible engine) installed in exactly this way.
- On servers, engineers install Docker Engine from the official repository, keep it patched, and restrict who can reach the socket, because socket access means root access.
- `docker info` is one of the first commands to run when a build machine "behaves strangely": it shows the engine version, storage location, and resources.

Key Takeaways

- Windows/macOS: Docker Desktop (WSL 2 on Windows). Linux: Docker Engine from Docker's repository.
- Verify with `docker version`, `docker compose version` and `docker run hello-world`.
- The `docker` group is equivalent to root on that machine.
- Use WSL 2 on Windows; in Git Bash set `MSYS_NO_PATHCONV=1`; in PowerShell use `${PWD}`.
- Next: [03 · Images](#).

Images

Lesson 03 · about 30 minutes · you need: 02 · [Installation](#) done

What is it?

A Docker **image** is a read-only package that contains:

- a filesystem: the files of a small operating-system userland (for example Alpine or Debian), plus the language runtime, libraries and your application;
- metadata: which command to start, which user to run as, default environment variables, which port the app listens on.

An image never runs by itself. You start a **container** from it. One image can start many containers, just like one recipe can be cooked many times.

IMAGE (read-only)		CONTAINERS (running instances)
+-----+		+-----+
nginx:1.30-alpine	docker run	web (running)
- Alpine Linux files	----->	
- nginx binary		+-----+
- default config	docker run	+-----+
- CMD: nginx -g daemon off	----->	web2 (running)
+-----+		+-----+

Why do we need it?

An image is the **unit of delivery** in the container world. Instead of "install Python 3.14, then pip install these packages, then copy these files", you ship one image that already contains all of that. The same image runs on your laptop, on a colleague's laptop, and on a server.

How does it work?

Image names

A full image name has four parts. Docker fills in defaults for the parts you leave out:

```
docker.io / library / nginx : 1.30-alpine
-----
registry namespace repo tag

nginx -> docker.io/library/nginx:latest
nginx:1.30-alpine -> docker.io/library/nginx:1.30-alpine
sufibaba6629/docker-from-zero-web:1.0.0
-> docker.io/sufibaba6629/docker-from-zero-web:1.0.0
```

- **registry:** where the image lives (`docker.io` = Docker Hub, the default).
- **namespace:** the account. `library` is reserved for **official images** maintained with Docker.
- **repository:** the image's name.

- **tag**: a label for a version. If you omit it, Docker uses `latest`.

Tags versus digests

A **tag** is a movable label: the owner can point `nginx:1.30-alpine` at a newer build tomorrow (for example after a security fix). A **digest** (`sha256:...`) identifies one exact image content and never changes.

`latest` is just a tag name; it does **not** mean "newest". It means "whatever the owner tagged `latest`". That is why this course always names a version (`nginx:1.30-alpine`). The one exception is `hello-world`, which has no meaningful versions.

Layers

An image is a stack of read-only **layers**. Each layer is a set of file changes. Images that share a base share those layers on disk, so ten Alpine-based images do not store Alpine ten times. Lesson [10 · Image Layers](#) goes deeper.

```
+-----+ <- your app files
+-----+ <- pip install flask
+-----+ <- python runtime
+-----+ <- debian slim base
```

Prerequisites

- A working Docker installation (`docker run hello-world` works).

Hands-on Lab

Download (pull) an image from Docker Hub without starting it:

```
docker pull nginx:1.30-alpine
```

```
1.30-alpine: Pulling from library/nginx
Digest: sha256:0985e772fb9f729e6fa0980da05fca5d9c468e870eed43071545afa9d2e27d94
Status: Image is up to date for nginx:1.30-alpine
docker.io/library/nginx:1.30-alpine
```

List the images stored on your computer:

```
docker images nginx
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
nginx:1.30-alpine	0985e772fb9f	93.6MB	27MB	

The full step-by-step version is part of [labs/01-first-container](#).

Expected Result

- `docker pull` prints one line per layer (`Pull complete` or `Download complete`), then a `Digest: sha256:...` line and `Status: Downloaded newer image for nginx:1.30-alpine`.

- `docker images nginx` shows one line per image: **IMAGE** (`nginx:1.30-alpine`), **ID** (the first 12 characters of the image's own hash), **DISK USAGE**, **CONTENT SIZE** (what was downloaded) and **EXTRA** (`U` when a container uses it). Docker versions before 29, and `docker images --format table`, show the classic columns REPOSITORY, TAG, IMAGE ID, CREATED (when the image was built by its maintainer) and SIZE.

Experiment

Pull the same image again. Docker compares digests, sees that it already has it, and downloads nothing:

```
docker pull nginx:1.30-alpine
```

Pull a second, unrelated image and compare sizes. `alpine` is a complete (tiny) Linux userland:

```
docker pull -q alpine:3.24
docker images --format 'table {{.Repository}}:{{.Tag}}\t{{.Size}}' | grep -E 'alpine|REPOSITORY'
```

```
docker.io/library/alpine:3.24
REPOSITORY:TAG          SIZE
nginx:1.30-alpine       93.6MB
postgres:18-alpine      433MB
alpine:3.24             13MB
```

Look at how an image was configured. `docker image inspect` prints its metadata as JSON; the `--format` option picks out single fields (here: the command and the exposed ports):

```
docker image inspect nginx:1.30-alpine --format 'CMD: {{.Config.Cmd}} ports: {{.Config.ExposedPorts}}'
```

```
CMD: [nginx -g daemon off;] ports: map[80/tcp:{}]
```

Break It

Ask for a tag that does not exist:

```
docker pull nginx:9.99-alpine
```

Troubleshoot It

- **Observe:** the pull fails with a message that the manifest for `nginx:9.99-alpine` is **not found** (wording differs slightly between Docker versions).
- **Investigate:** the repository exists (it is `nginx`), so the problem is the tag. Check the available tags on the image's page on Docker Hub (https://hub.docker.com/_/nginx, tab "Tags").
- **Root cause:** a tag that was never published.
- **Fix:** use an existing tag, for example `nginx:1.30-alpine`.
- **Verify:** `docker pull nginx:1.30-alpine` ends with `Status: ... nginx:1.30-alpine`.

Common Mistakes

- Using `latest` and getting a different version next month without noticing.
- Thinking `CREATED` in `docker images` is the download date. It is the build date.
- Deleting an image that a container still uses: Docker refuses (`image is being used by stopped container`). Remove the container first.
- Pulling random images from unknown accounts. Anyone can publish to Docker Hub.

Best Practices

- Pin a version tag (`postgres:18-alpine` , `python:3.14-slim`). For maximum reproducibility, pin the digest too: `nginx:1.30-alpine@sha256:...`
- Prefer **Docker Official Images** (`library/...`) and **Verified Publisher** images.
- Prefer small variants (`-alpine` , `-slim`) when they work for your app: less to download, fewer packages that could contain vulnerabilities.
- Remove images you no longer need (`docker rmi` , lesson 05 · [Docker CLI](#)).

Challenge

Pull `busybox:1.37` and find out its size and the command it runs by default, without starting a container.

Solution

▼ Show the solution

```
docker pull -q busybox:1.37
docker images busybox:1.37 --format '{{.Repository}}:{{.Tag}} is {{.Size}}'
docker image inspect busybox:1.37 --format 'default command: {{.Config.Cmd}}'
```

```
docker.io/library/busybox:1.37
busybox:1.37 is 6.77MB
default command: [sh]
```

The default command is a shell (`sh`). Started without a terminal, a shell has nothing to do and exits immediately; lesson 06 · [Container Lifecycle](#) explains why.

Verification

- [] You can explain registry / namespace / repository / tag.
- [] You can pull an image and list your images.
- [] You can explain why `latest` is risky.
- [] You can read an image's default command with `docker image inspect` .

Real-World Usage

- Teams publish their applications as images to a registry; deployments pull a specific tag (often a version number or a git commit) so they know exactly what runs.
- Security teams scan images for known vulnerabilities and require trusted base images.
- Pinning tags and digests keeps builds reproducible: the build next year uses the same base.

Key Takeaways

- An image is a read-only package of files plus metadata; containers are started from it.
- Name format: `registry/namespace/repository:tag`; defaults are `docker.io`, `library`, `latest`.
- Tags can move, digests cannot. Never rely on `latest`.
- Images are made of shared layers.
- Next: [04 · Containers](#).

Containers

Lesson 04 · about 30 minutes · you need: 03 · Images

What is it?

A **container** is a running (or stopped) instance of an image. Technically it is one or more ordinary Linux processes that the kernel isolates:

- it sees **its own filesystem**: the image's read-only layers plus one thin **writable layer** that belongs only to this container;
- it sees **its own processes**: inside, the main process is PID 1;
- it has **its own hostname and network interface** (and IP address);
- it can be given **limits** for CPU and memory.

```
container "web"                container "web2"
+-----+                    +-----+
| writable layer (own) |      | writable layer (own) |
+-----+                    +-----+
|                               |
|   nginx:1.30-alpine image layers (read-only, SHARED)   |
+-----+                    +-----+
```

Why do we need it?

The image is the package; the container is where the work happens. Containers give you:

- **isolation**: an app in one container cannot see or break files of another;
- **speed**: starting a container takes about a second;
- **disposability**: you can delete a container and create a fresh one from the same image at any time. Nothing lingers on your computer.

How does it work?

`docker run` is a shortcut for four steps:

```
docker run -d --name web nginx:1.30-alpine
|
+-- 1. image present? if not: docker pull nginx:1.30-alpine
+-- 2. docker create (writable layer, network, settings) -> state: Created
+-- 3. docker start (start the main process) -> state: Up / running
+-- 4. -d: return immediately and print the container ID
    (without -d: attach your terminal to the container's output)
```

Each container gets:

Property	Example	Where you see it
Container ID	64 hex characters, shown short as 12	<code>docker ps</code> (CONTAINER ID)

Property	Example	Where you see it
Name	web (yours) or a random one like eager_turing	docker ps (NAMES)
Main process	nginx -g 'daemon off;'	docker ps (COMMAND)
State	Up, Exited (0), Exited (137) ...	docker ps -a (STATUS)

The golden rule: a container lives exactly as long as its **main process**. When that process ends, the container stops. Lesson 06 · [Container Lifecycle](#) is built around this rule.

Prerequisites

- nginx:1.30-alpine pulled, or an internet connection (Docker pulls it automatically).

Hands-on Lab

Start nginx in the background (`-d` = detached) with a name you choose:

```
docker run -d --name web nginx:1.30-alpine
```

```
d4f9d0325f28326214307dd7ebb528965825c9853bf470da0b5d30d141c08fd1
```

The long hexadecimal string is the full **container ID**. List running containers:

```
docker ps
```

```
CONTAINER ID   IMAGE          COMMAND                  CREATED          STATUS
PORTS         NAMES
d4f9d0325f28   nginx:1.30-alpine  "/docker-entrypoint...."  Less than a second ago  Up Less than a
second        80/tcp         web
```

Start a second container from the **same** image. It is completely independent:

```
docker run -d --name web2 nginx:1.30-alpine
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

Prove the isolation: create a file in `web` and look for it in `web2`:

```
docker exec web sh -c 'echo hello > /tmp/only-in-web.txt'
docker exec web ls /tmp
```

```
docker exec web2 ls /tmp
```

The full hands-on lab is [labs/01-first-container](#).

Expected Result

`docker ps` prints one row per running container with these columns:

Column	Meaning
CONTAINER ID	the short ID (first 12 characters)
IMAGE	the image it was started from
COMMAND	the main process (shortened)
CREATED	when the container was created
STATUS	Up 5 seconds , Exited (0) 2 minutes ago , ...
PORTS	published ports (empty here: we published none)
NAMES	the name you gave with <code>--name</code>

`web` has `/tmp/only-in-web.txt` ; `web2` does not. Same image, separate writable layers.

Experiment

Each container has its own hostname. By default it is the short container ID:

```
docker exec web hostname
docker exec web2 hostname
```

```
d4f9d0325f28
58c93d2bb5b0
```

Compare with the CONTAINER ID column of `docker ps` : they match.

Break It

Try to create another container named `web` :

```
docker run -d --name web nginx:1.30-alpine
```

Troubleshoot It

- **Observe:** Conflict. The container name `"/web"` is already in use by container `"<id>"`.
- **Investigate:** `docker ps -a --filter name=web` shows the existing container with that name (it might even be stopped, which is why `docker ps` alone sometimes hides it).
- **Root cause:** container names are unique on one Docker engine.
- **Fix:** choose another name, or remove the old container first (`docker rm -f web`).
- **Verify:**

```
docker rm -f web
docker run -d --name web nginx:1.30-alpine
docker ps --filter name=web
```

Common Mistakes

- Expecting changes inside a container to appear in other containers or in the image. They live only in that container's writable layer, and they are deleted with the container.
- Forgetting `-d` and wondering why the terminal "hangs": without `-d`, your terminal shows the container's output until you press Ctrl+C (which also stops nginx).
- Using `docker ps` to look for a stopped container. Use `docker ps -a`.
- Giving names with one character: Docker rejects them; use at least two.

Best Practices

- Always give containers you work with a **name** (`--name web`); typing `web` beats copying IDs.
- Treat containers as **disposable**: never store important data only in a container's writable layer (lesson 12 · Volumes).
- One main concern per container (a web server, a database, an API), not everything in one.

Challenge

Start three nginx containers called `site1`, `site2`, `site3`, show only their names and statuses in a table, then remove all three with **one** command.

Solution

▼ Show the solution

```
docker run -d --name site1 nginx:1.30-alpine
docker run -d --name site2 nginx:1.30-alpine
docker run -d --name site3 nginx:1.30-alpine
docker ps --filter name=site --format 'table {{.Names}}\t{{.Status}}'
```

`docker rm -f` accepts several names at once (`-f` stops running containers before removing):

```
docker rm -f site1 site2 site3
```

```
docker ps --format '{{.Names}}'
```

Verification

- [] You can start a named container in the background.
- [] You can read every column of `docker ps`.
- [] You can explain why a file created in one container is not in another.
- [] You know that a container stops when its main process stops.

Clean up:

```
docker rm -f web web2
```

Real-World Usage

- In production, each service (web server, API, worker, database) runs as one or more containers started from versioned images.
- Scaling out means "start more containers from the same image".
- Because containers are disposable, a broken one is often simply replaced by a fresh one.

Key Takeaways

- A container = image layers (shared, read-only) + its own writable layer + isolated processes.
- `docker run` = pull (if needed) + create + start.
- Names must be unique; IDs are generated.
- A container lives as long as its main process.
- Next: [05 · Docker CLI](#).

Docker CLI

Lesson 05 · about 45 minutes · you need: 04 · [Containers](#)

What is it?

The Docker CLI is the `docker` command. It has one sub-command per job. You do not need to memorize them all: about twenty cover 95% of daily work, and every one of them has `--help`.

```
docker <object> <action> [options] [arguments]
docker container ls -a (long form)
docker ps -a (classic short form, same thing)
```

Why do we need it?

Everything you do with Docker (starting, stopping, debugging, cleaning up) happens through these commands. Knowing what each one is **for** lets you investigate problems instead of guessing.

How does it work?

Every command is a request to the Docker Engine. The engine answers with data (lists, JSON, logs) or performs an action.

Command reference

Command	What it does	Syntax · important options	Real-world use
<code>docker version</code>	Client and engine versions	<code>--format '{{.Server.Version}}'</code>	First check when something "does not work"
<code>docker info</code>	Engine-wide details: containers, images, storage, CPUs, memory	<code>--format</code>	Check resources and setup of a build machine
<code>docker images</code>	List local images	<code>-a</code> , <code>--filter dangling=true</code> , <code>--format</code>	See what takes disk space
<code>docker pull</code>	Download an image	<code>NAME:TAG</code> , <code>-q</code> quiet	Pre-fetch images before an offline demo
<code>docker run</code>	Create and start a container	<code>-d</code> background, <code>--name</code> , <code>-p</code> ports, <code>-e</code> env, <code>-v</code> volumes, <code>--rm</code> delete on exit, <code>-it</code> interactive	Start any service
<code>docker ps</code>	List running containers	<code>-a</code> all, <code>-q</code> IDs only, <code>--filter</code> , <code>--format</code>	"What is running right now?"
<code>docker stop</code>	Ask the main process to stop (stop signal, usually SIGTERM; SIGKILL after 10 s)	<code>-t</code> seconds	Graceful shutdown

Command	What it does	Syntax · important options	Real-world use
<code>docker start</code>	Start a stopped container again	<code>-a</code> attach	Resume a stopped container with its data
<code>docker restart</code>	stop + start	<code>-t</code>	Reload after a configuration change
<code>docker rm</code>	Delete a stopped container	<code>-f</code> force (stop first), <code>-v</code> with anonymous volumes	Clean up
<code>docker rmi</code>	Delete an image	<code>-f</code>	Free disk space
<code>docker logs</code>	Show what the main process printed (stdout/stderr)	<code>-f</code> follow, <code>--tail N</code> , <code>--since 5m</code> , <code>-t</code> timestamps	First step of almost every investigation
<code>docker exec</code>	Run an extra command inside a running container	<code>-it</code> for a shell, <code>-u</code> user, <code>-e</code> env	Look around inside: files, processes, network
<code>docker inspect</code>	Every detail of a container/image/network/volume as JSON	<code>--format '{{...}}'</code>	Find IP, mounts, env, exit code, health
<code>docker stats</code>	Live CPU / memory / network / disk I/O per container	<code>--no-stream</code> one snapshot	Find the container that eats memory
<code>docker top</code>	Processes running inside a container	<code>docker top NAME</code>	See what is really running
<code>docker cp</code>	Copy files between host and container	<code>docker cp NAME:/path ./here</code>	Grab a log or config file

Every command answers `--help`, for example `docker run --help`.

Prerequisites

- Docker works; `nginx:1.30-alpine` can be pulled.

Hands-on Lab

Start something to work with:

```
docker run -d --name web -p 8080:80 nginx:1.30-alpine
docker ps --filter name=web
```

logs — what did nginx print? Each line comes from the container's stdout/stderr:

```
docker logs web
```

```
...
2026/10/04 05:02:20 [notice] 1#1: start worker process 39
2026/10/04 05:02:20 [notice] 1#1: start worker process 40
2026/10/04 05:02:20 [notice] 1#1: start worker process 41
2026/10/04 05:02:20 [notice] 1#1: start worker process 42
2026/10/04 05:02:20 [notice] 1#1: start worker process 43
```

exec — run a command inside the running container:

```
docker exec web nginx -v
docker exec web cat /etc/os-release
```

```
nginx version: nginx/1.30.5
NAME="Alpine Linux"
ID=alpine
VERSION_ID=3.24.2
PRETTY_NAME="Alpine Linux v3.24"
HOME_URL="https://alpinelinux.org/"
BUG_REPORT_URL="https://gitlab.alpinelinux.org/alpine/aports/-/issues"
```

inspect — the container's details as JSON; `--format` picks one value:

```
docker inspect web --format 'state={{.State.Status}} ip={{range .NetworkSettings.Networks}}
{{.IPAddress}}{{end}} started={{.State.StartedAt}}'
```

```
state=running ip=172.17.0.2 started=2026-10-04T05:02:20.170081961Z
```

stats — one snapshot of resource use (without `--no-stream` it updates live until Ctrl+C):

```
docker stats --no-stream web
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
c80c057b4b65	web	0.00%	11.64MiB / 15.35GiB	0.07%	1.17kB / 126B	0B / 8.19kB	15

top — the processes inside the container:

```
docker top web
```

UID	PID	PPID	C	STIME	TTY
TIME	CMD				
root	1685514	1685490	0	05:02	?
00:00:00	nginx: master process nginx -g daemon off;				
statd	1685558	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685559	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685560	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685561	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685562	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685563	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685564	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685565	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685566	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685567	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685568	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685569	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685570	1685514	0	05:02	?
00:00:00	nginx: worker process				
statd	1685571	1685514	0	05:02	?
00:00:00	nginx: worker process				

cp — copy nginx's default page out of the container to your computer:

```
docker cp web:/usr/share/nginx/html/index.html ./nginx-default.html
grep -o '<title>.*</title>' nginx-default.html
rm nginx-default.html
```

The full guided lab is [labs/02-docker-cli](#).

Expected Result

- `docker logs web` ends with nginx's start-up lines (`start worker process ...`).
- `docker exec web nginx -v` prints the nginx version; `/etc/os-release` says **Alpine Linux**: the container has its own userland, whatever your host runs.
- `docker inspect --format` prints `state=running` and an IP address (typically `172.17.0.x`).
- `docker stats --no-stream` shows CPU %, MEM USAGE / LIMIT, NET I/O, BLOCK I/O and PIDS.
- `docker top web` shows the nginx master process and its worker processes.

Experiment

Make a request, then look at the logs again: nginx writes an access-log line for every request.

```
curl -s -o /dev/null http://localhost:8080
docker logs --tail 1 web
```

`--tail 1` shows only the last line; `--since 1m` would show the last minute; `-f` would keep following new lines until you press Ctrl+C:

```
docker logs -f web
```

Break It

`exec` only works on **running** containers. Stop `web` and try:

```
docker stop web
docker exec web nginx -v
```

Troubleshoot It

- **Observe:** Error response from daemon: container ... is not running .
- **Investigate:** `docker ps -a --filter name=web` shows `Exited (0)` : we stopped it. Logs are still readable on stopped containers: `docker logs web` .
- **Root cause:** `exec` starts an extra process inside the container; a stopped container has no running environment to start it in.
- **Fix:** `docker start web` .
- **Verify:**

```
docker start web
docker exec web nginx -v
```

Common Mistakes

- Confusing `docker run` (creates a **new** container) with `docker start` (restarts an **existing** one). Running `docker run` again and again leaves many stopped containers behind.
- Confusing `docker rm` (containers) with `docker rmi` (images).
- Using `docker exec -it ... bash` on Alpine images: they have `sh` , not `bash` .
- Reading `docker stats` without `--no-stream` in a script: it never returns.

Best Practices

- Start every investigation with `docker ps -a` and `docker logs <name>` .
- Use `--format` to get exactly the field you need instead of scrolling through JSON.
- Use `--rm` for one-off containers (`docker run --rm alpine:3.24 echo hi`) so they delete themselves.
- Use `docker <command> --help` instead of guessing options.

Challenge

Using only the CLI: find out **how many worker processes** nginx started inside `web`, and copy nginx's main configuration file `/etc/nginx/nginx.conf` to your current folder.

Solution

▼ Show the solution

```
docker top web | grep -c 'worker process'
docker top web
```

```
docker cp web:/etc/nginx/nginx.conf ./nginx.conf
grep worker_processes nginx.conf
rm nginx.conf
```

`worker_processes auto;` means one worker per CPU the container can see, so the count depends on your machine.

Verification

- [] You can read logs (`docker logs` , `--tail` , `-f`).
- [] You can run a command inside a container (`docker exec`).
- [] You can pick one value out of `docker inspect` with `--format` .
- [] You can see resource usage and processes (`docker stats --no-stream` , `docker top`).
- [] You can copy a file out of a container (`docker cp`).

Clean up:

```
docker rm -f web
```

Real-World Usage

- On-call engineers start incidents with `docker ps -a` , `docker logs --since 10m` and `docker inspect` to see exit codes and restart counts.
- `docker exec` is how you check, from inside, whether a service can reach its database.
- `docker cp` grabs a crash dump or a generated report out of a container.

Key Takeaways

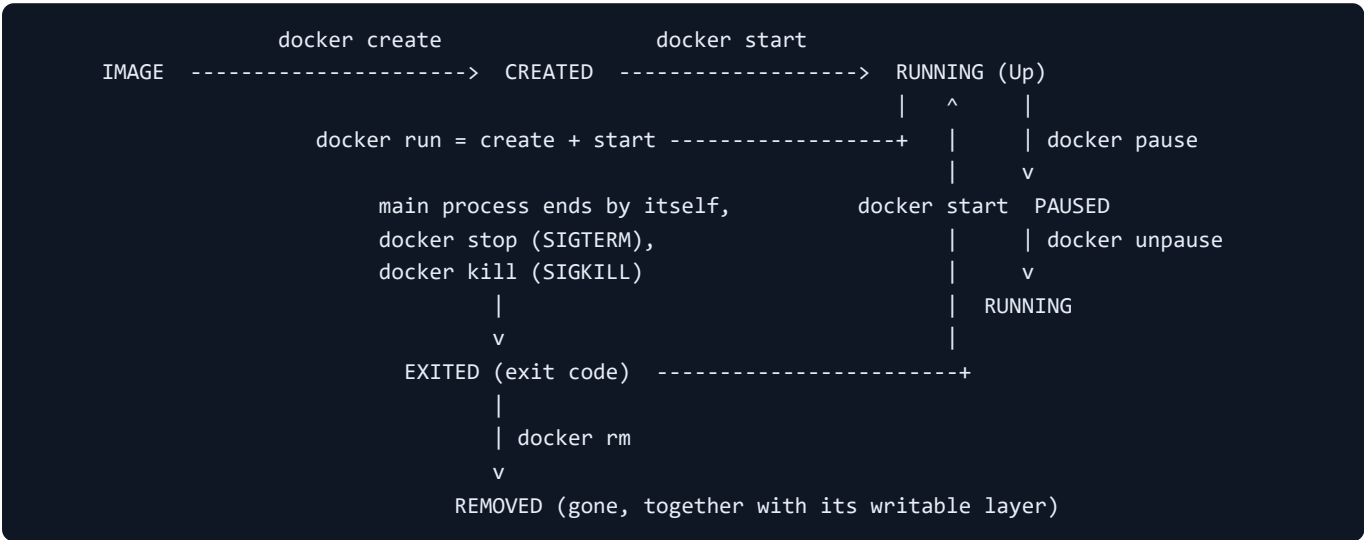
- `run` = new container; `start` = existing one; `rm` = containers; `rmi` = images.
- `logs` , `exec` , `inspect` , `stats` , `top` , `cp` are your investigation tools.
- `--format` and `--filter` turn long outputs into exact answers.
- Next: [06 · Container Lifecycle](#).

Container Lifecycle

Lesson 06 · about 40 minutes · you need: 05 · Docker CLI

What is it?

Every container moves through a small set of **states**. Knowing them tells you what you can do with a container right now, and why it is (or is not) running.



Why do we need it?

"My container is not running" is the most common Docker problem. The state and the **exit code** tell you why:

STATUS in <code>docker ps -a</code>	Meaning
Created	created, never started
Up 3 minutes	the main process is running
Up 3 minutes (healthy)	running, and its health check passes
Exited (0)	the main process finished normally
Exited (1) , (2) , (3) ...	the process failed; the number comes from the app itself
Exited (137)	killed with SIGKILL: <code>docker kill</code> , or out of memory (137 = 128 + 9)
Exited (143)	ended by SIGTERM and the app reported it (143 = 128 + 15)
Restarting	crashed and a restart policy is starting it again

How does it work?

Image, container, process

```

IMAGE  (read-only template, on disk)
|
| docker run
v
CONTAINER (writable layer + settings + network)
|
| runs
v
MAIN PROCESS (PID 1 inside the container, e.g. nginx, python app.py, sleep 30)

```

The container **is not** the process, but it is **tied** to it. When PID 1 exits, the container stops. Docker does not keep an "empty" container running.

How docker stop works

`docker stop` sends a stop signal ("please shut down") to PID 1, waits up to 10 seconds, then sends **SIGKILL** ("die now"). The stop signal is **SIGTERM** unless the image chooses another one: the nginx image uses SIGQUIT, nginx's own "graceful shutdown" signal (you can see it with `docker image inspect nginx:1.30-alpine --format '{{.Config.StopSignal}}'`). `docker kill` sends SIGKILL immediately. Well-behaved apps (nginx, PostgreSQL) handle their stop signal and shut down cleanly with exit code 0.

Why containers stop

A container stops when its main process ends. Common reasons:

1. the job is done (`echo hello` prints and exits);
2. the program is a shell with no input (`docker run ubuntu:26.04 runs bash` , which has no terminal attached, reads end-of-input, and exits immediately);
3. the program crashed (wrong command, missing file, missing setting);
4. someone stopped or killed it, or it ran out of memory.

Prerequisites

- Images `nginx:1.30-alpine` , `alpine:3.24` and `ubuntu:26.04` (Docker pulls them when needed).

Hands-on Lab

Walk through every state by hand. **Create** without starting:

```

docker create --name demo nginx:1.30-alpine
docker ps -a --filter name=demo --format 'table {{.Names}}\t{{.Status}}'

```

```

d4d7327177dfcd9c4d5a6231c6ad587c9997aac5e8b9eb2076f9fb3d6a148704
NAMES      STATUS
demo       Created

```

Start it:

```

docker start demo
docker ps -a --filter name=demo --format 'table {{.Names}}\t{{.Status}}'

```

```
demo
NAMES      STATUS
demo       Up Less than a second
```

Stop it gracefully (stop signal, then SIGKILL only if needed):

```
docker stop demo
docker ps -a --filter name=demo --format 'table {{.Names}}\t{{.Status}}'
```

```
demo
NAMES      STATUS
demo       Exited (0) Less than a second ago
```

Start it again, then **kill** it (SIGKILL):

```
docker start demo
docker kill demo
docker ps -a --filter name=demo --format 'table {{.Names}}\t{{.Status}}'
```

```
demo
demo
NAMES      STATUS
demo       Exited (137) Less than a second ago
```

Remove it:

```
docker rm demo
```

```
docker ps -a --format '{{.Names}}'
```

Interactive containers

`-i` keeps input open, `-t` gives you a terminal. Together they let you work inside a container like on a remote machine. Type `exit` (or Ctrl+D) to leave; leaving ends `bash`, so the container stops:

```
docker run -it --name shell ubuntu:26.04 bash
```

Inside, try `ls /`, `ps`, `env`, `hostname`, `cat /etc/os-release`. You can run the same commands without a terminal, which is how they are tested here:

```
docker run --rm ubuntu:26.04 cat /etc/os-release
```

```
PRETTY_NAME="Ubuntu 26.04.1 LTS"
NAME="Ubuntu"
VERSION_ID="26.04"
VERSION="26.04.1 LTS (Resolute Raccoon)"
VERSION_CODENAME=resolute
ID=ubuntu
...
```

The full lab is [labs/03-container-lifecycle](#).

Expected Result

- Created after `docker create`, Up ... after `docker start`.
- Exited (0) after `docker stop`: nginx shut down cleanly on its stop signal.
- Exited (137) after `docker kill`: 128 + 9 (SIGKILL).
- After `docker rm`, the container is gone from `docker ps -a`.

Experiment

A container that only lives as long as its job:

```
docker run -d --name sleeper alpine:3.24 sleep 5
docker ps --filter name=sleeper --format '{{.Names}}: {{.Status}}'
```

Wait more than 5 seconds, then look again: `sleep` finished, so the container exited with 0.

```
docker ps -a --filter name=sleeper --format '{{.Names}}: {{.Status}}'
```

And the classic beginner surprise: Ubuntu "does not start".

```
docker run --name ubuntu-test ubuntu:26.04
docker ps -a --filter name=ubuntu-test --format '{{.Names}}: {{.Status}}'
```

It **did** start. Its main process is `bash`, without a terminal it has nothing to read, so it exits at once. To keep a container busy, give it a long-running process (a server, or `sleep infinity` for experiments).

Break It

Run a container whose main process fails:

```
docker run --name broken alpine:3.24 sh -c 'echo "starting..."; ls /does-not-exist'
```

Troubleshoot It

- **Observe:** the command printed `starting...` and an `ls` error, and returned to the prompt.
- **Investigate:** the state and the exit code, then the logs:

```
docker ps -a --filter name=broken --format '{{.Names}}: {{.Status}}'
docker inspect broken --format 'exit code: {{.State.ExitCode}}'
docker logs broken
```

- **Root cause:** `ls` failed (exit code 1), PID 1 ended, the container stopped.
- **Fix:** run a command that works (in a real app: fix the command, file or setting the logs name).
- **Verify:**

```
docker rm broken
docker run --name fixed alpine:3.24 sh -c 'echo "starting..."; ls /'
docker ps -a --filter name=fixed --format '{{.Names}}: {{.Status}}'
```

Common Mistakes

- Thinking `Exited (0)` means "crashed". It means "finished normally": often the command simply had nothing long-running to do.
- Looking only at `docker ps`: stopped containers are only in `docker ps -a`.
- Using `docker kill` by habit: it skips the clean shutdown (databases may need recovery afterwards).
- Expecting `docker start` to apply new options. Options (ports, env, volumes) are fixed at `docker create` / `run` time; to change them, remove the container and run a new one.

Best Practices

- Read the exit code first, then the logs: together they explain almost every stopped container.
- Use `docker stop` (graceful) rather than `docker kill`.
- Keep the main process in the foreground (nginx uses `daemon off;` for exactly this reason).
- Use `--rm` for one-off experiments so stopped containers do not pile up.

Challenge

Start an `alpine:3.24` container named `timer` that stays running for exactly 20 seconds. While it runs, use `docker exec` to print its hostname. Then prove it stopped by itself with exit code 0.

Solution

▼ Show the solution

```
docker run -d --name timer alpine:3.24 sleep 20
docker exec timer hostname
docker ps --filter name=timer --format '{{.Names}}: {{.Status}}'
```

After 20 seconds:

```
docker inspect timer --format 'state={{.State.Status}} exit code {{.State.ExitCode}}'
```

Verification

- [] You can name every state and the command that leads to it.
- [] You can explain `Exited (0)`, `(1)` and `(137)`.
- [] You can explain why `docker run ubuntu:26.04` exits immediately.
- [] You know the difference between `docker stop` and `docker kill`.

Clean up:

```
docker rm -f sleeper ubuntu-test fixed timer
```

Real-World Usage

- `Exited (137)` with `OOMKilled: true` is how you recognise a service that needs more memory (lesson [18 · Resource Management](#)).
- Restart policies (`--restart unless-stopped`) bring crashed services back; the restart count in `docker inspect` tells you how often it happened.
- Graceful SIGTERM handling is what makes zero-downtime deployments possible on any container platform.

Key Takeaways

- States: Created → Up → Exited → removed (plus Paused, Restarting).
- A container lives exactly as long as its main process (PID 1).
- Exit codes tell the story: 0 normal, 1–127 app errors, 137 killed, 143 terminated.
- `stop` = stop signal (SIGTERM by default) then SIGKILL; `kill` = SIGKILL now.
- Next: [07 · Ports](#).

Ports

Lesson 07 · about 30 minutes · you need: 04 · Containers

What is it?

A container has its **own network interface** with a private IP address. A program listening on port 80 inside the container is **not** reachable from your browser automatically. **Port publishing** (`-p`) tells Docker: "forward traffic that arrives on this port of my computer to that port of the container".

```
-p 8080:80
HOST : CONTAINER
```

```
Browser  -> localhost:8080  -> Docker  -> container 172.17.0.2:80  -> nginx
      (your computer)      (forward)   (private, inside Docker)
```

Why do we need it?

Containers are isolated on purpose. Publishing is how you deliberately open exactly the ports you want: the web server, yes; the database, usually no. It also lets you run several containers that all listen on port 80 inside, by giving each a different port on the host.

How does it work?

Option	Meaning
<code>-p 8080:80</code>	host port 8080 → container port 80, on all host interfaces
<code>-p 127.0.0.1:8080:80</code>	only reachable from your own computer, not from the network
<code>-p 80</code>	container port 80 → a random free host port (see it with <code>docker port</code>)
<code>-P</code>	publish every port the image <code>EXPOSE</code> s, to random host ports
<code>EXPOSE 80</code> (in a Dockerfile)	documentation only : says which port the app uses, publishes nothing

```
Your computer (host)
+-----+
| port 8080  +-----+
| port 8081  +-----+ |
|               | | Docker network (172.17.0.0/16) |
|               | +-> web  172.17.0.2:80  (nginx)   |
|               +----> web2 172.17.0.3:80  (nginx)   |
| (nothing published) ---> db   172.17.0.4:5432 (private) |
+-----+
```

Two rules decide whether a request arrives:

1. the **host port** must be free (only one program per port on your computer);
2. the **container port** must be the port the app really listens on.

Prerequisites

- `curl` in your terminal (WSL, Linux, macOS, Git Bash; PowerShell has `curl.exe`).

Hands-on Lab

Publish nginx's port 80 on port 8080 of your computer:

```
docker run -d --name web -p 8080:80 nginx:1.30-alpine
docker ps --filter name=web --format 'table {{.Names}}\t{{.Ports}}'
```

```
a0b7facb3c3b7b31889c369bd741056238db1cf2a6db28ff144edf758b1f2a08
NAMES      PORTS
web        0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
```

Open <http://localhost:8080> in your browser, or ask with curl:

```
curl -s http://localhost:8080 | grep -o '<title>.*</title>'
```

`docker port` lists the mappings of one container:

```
docker port web
```

```
80/tcp -> 0.0.0.0:8080
80/tcp -> [::]:8080
```

The full lab, with port conflicts and a "running but unreachable" case, is [labs/04-port-mapping](#).

Expected Result

- The PORTS column shows `0.0.0.0:8080->80/tcp` (and often `[::]:8080->80/tcp` for IPv6).
- curl prints `<title>Welcome to nginx!</title>`.
- `docker port web` prints `80/tcp -> 0.0.0.0:8080`.

Experiment

Run a second nginx on a **different host port**. Inside, both listen on 80, and that is fine:

```
docker run -d --name web2 -p 8081:80 nginx:1.30-alpine
```

```
curl -s http://localhost:8081 | grep -o '<title>.*</title>'
```

Let Docker choose a free host port with `-p 80`:

```
docker run -d --name web3 -p 80 nginx:1.30-alpine
docker port web3
```

```
a734d54db395b18807dc0527e1dada807dcd1acf281e8cbad0060a14f0f75dad
80/tcp -> 0.0.0.0:50958
80/tcp -> [::]:50958
```

Break It

Break 1 · the host port is already taken. Port 8080 belongs to `web`:

```
docker run -d --name web4 -p 8080:80 nginx:1.30-alpine
```

Break 2 · wrong container port. nginx listens on 80, but we forward to 8080 inside:

```
docker run -d --name wrong -p 8082:8080 nginx:1.30-alpine
docker ps --filter name=wrong --format '{{.Names}} {{.Status}} {{.Ports}}'
```

The container is **running**, yet:

```
curl -sS --max-time 5 http://localhost:8082
```

Troubleshoot It

Break 1: the error ends with `Bind for 0.0.0.0:8080 failed: port is already allocated`. Investigate who owns the port:

```
docker ps --filter publish=8080 --format '{{.Names}} uses {{.Ports}}'
```

Root cause: two containers cannot share one **host** port. Fix: choose another host port (`-p 8083:80`) or stop the other container. Note that Docker already **created** `web4` before the port failed; remove the leftover with `docker rm web4`.

```
docker rm web4
docker run -d --name web4 -p 8083:80 nginx:1.30-alpine
```

```
curl -s http://localhost:8083 | grep -o '<title>.*</title>'
```

If the port is used by a program **outside** Docker, `docker ps` will not show it. On Linux/WSL use `sudo ss -ltnp | grep 8080`, on Windows `netstat -ano | findstr :8080`, on macOS `lsof -i :8080`.

Break 2: curl fails (connection reset, empty reply, or refused, depending on your system). Do not guess, collect evidence:

1. Is the container running? `docker ps` says **Up**. So it is not a crash.
2. What is published? `8082->8080/tcp`.
3. What does the app listen on? Ask the image and the process:

```
docker image inspect nginx:1.30-alpine --format 'image exposes: {{.Config.ExposedPorts}}'
docker exec wrong grep -m1 listen /etc/nginx/conf.d/default.conf
```

Root cause: we forward to container port 8080, but nginx listens on 80. Fix: recreate the container with the right container port (options cannot be changed on an existing container):

```
docker rm -f wrong
docker run -d --name wrong -p 8082:80 nginx:1.30-alpine
```

```
curl -s http://localhost:8082 | grep -o '<title>.*</title>'
```

Common Mistakes

- Writing the ports the wrong way round. It is always `HOST:CONTAINER` .
- Thinking `EXPOSE` publishes a port. It does not; only `-p / -P` (or Compose `ports:`) do.
- Using `-p 80:80` on Linux/WSL where port 80 may be used by another web server, or needs root.
- An app inside the container listening on `127.0.0.1` only: it then refuses traffic coming from Docker's port forwarding. Apps in containers must listen on `0.0.0.0` (scenario 07 · [running but not reachable](#)).

Best Practices

- Publish only what users must reach (web front end); keep databases and internal APIs unpublished.
- Use `-p 127.0.0.1:8080:80` for services that only you should reach on a shared network.
- Pick high host ports (8080, 8081, ...) for local development to avoid conflicts.
- Document the app's port with `EXPOSE` in your Dockerfile.

Challenge

Run two nginx containers, `blue` on host port 8091 and `green` on host port 8092, and prove with curl that both answer. Then find, with a Docker command, which container owns port 8092.

Solution

▼ Show the solution

```
docker run -d --name blue -p 8091:80 nginx:1.30-alpine
docker run -d --name green -p 8092:80 nginx:1.30-alpine
```

```
curl -s http://localhost:8091 | grep -o '<title>.*</title>'
curl -s http://localhost:8092 | grep -o '<title>.*</title>'
```

```
docker ps --filter publish=8092 --format '{{.Names}}'
```

Verification

- [] You can explain `HOST:CONTAINER` .
- [] You know `EXPOSE` is documentation and `-p` is publishing.
- [] You can find which container uses a port.

- [] You can diagnose "running but unreachable" with evidence, not guesses.

Clean up:

```
docker rm -f web web2 web3 web4 wrong blue green
```

Real-World Usage

- Locally, every developer runs the app on a published port (8080) and the database unpublished.
- On servers, usually only a reverse proxy or load balancer publishes 80/443; everything else talks over internal networks.
- "Port already allocated" is a daily message when several projects run on one laptop.

Key Takeaways

- `-p HOST:CONTAINER` forwards a host port to a container port.
- `EXPOSE` documents, `-p` publishes.
- One host port = one container; the container port must match what the app listens on.
- Investigate with `docker ps`, `docker port`, `docker ps --filter publish=`, `docker inspect`.
- Next: [08 · Environment Variables](#).

Environment Variables

Lesson 08 · about 30 minutes · you need: 07 · Ports

What is it?

An **environment variable** is a named value (`APP_ENV=production`) that the operating system hands to every program it starts. Docker lets you set environment variables for the main process of a container with `-e NAME=value` (or a file with `--env-file`). Programs read them to decide how to behave.

```
docker run -e APP_ENV=development -e GREETING="Hi" simple-app:1.0
      |               |
      v               v
      +-----+
      | container |
      | environment: APP_ENV=development |
      |               GREETING=Hi       |
      |               PATH=... (from the image) |
      | python app.py -> os.environ["APP_ENV"] |
      +-----+
```

Why do we need it?

You want **one image** for every environment. The code is the same in development, test and production; only the **configuration** differs: database host, log level, feature flags, a greeting text. Environment variables carry that configuration **into** the container at start time, so you never rebuild the image just to change a setting. This idea is part of the widely used "twelve-factor app" guidelines.

How does it work?

Values can come from three places. Later ones win:

1. ENV in the Dockerfile	ENV APP_ENV=production	(default, baked into the image)
2. --env-file on docker run	APP_ENV=staging (file)	(overrides 1)
3. -e on docker run	-e APP_ENV=development	(overrides 1 and 2)

Syntax	Meaning
<code>-e NAME=value</code>	set NAME to value
<code>-e NAME</code>	copy NAME's value from your terminal's environment
<code>--env-file app.env</code>	read many <code>NAME=value</code> lines from a file

The program must actually **read** the variable. `examples/simple-app/app.py` does:

```
APP_ENV = os.environ.get("APP_ENV", "development")
GREETING = os.environ.get("GREETING", "Hello from simple-app")
```

Prerequisites

- Lessons 04–07. This lesson also builds `examples/simple-app` (a few seconds; lesson 09 · [Dockerfile](#) explains how).

Hands-on Lab

See a container's environment. `env` prints every variable of the process:

```
docker run --rm -e APP_ENV=development -e TEAM=platform alpine:3.24 env
```

```
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=e1a60ee19fb5
APP_ENV=development
TEAM=platform
HOME=/root
```

Now with a real app. Build `simple-app` and run it twice with different configuration:

```
docker build -q -t simple-app:1.0 examples/simple-app
```

```
docker run -d --name prod -p 5000:5000 simple-app:1.0
```

```
curl -s http://localhost:5000
```

```
docker run -d --name dev -p 5001:5000 -e APP_ENV=development -e GREETING="Hi from the dev container"
simple-app:1.0
```

```
curl -s http://localhost:5001
```

Same image, two behaviours. The full lab is [labs/05-environment-variables](#).

Expected Result

- `env` lists `APP_ENV=development` and `TEAM=platform`, plus `PATH` and `HOSTNAME`, which Docker and the image always provide.
- `prod` answers with `environment: production` (the Dockerfile's `ENV` default).
- `dev` answers with `Hi from the dev container!` and `environment: development` (`-e` overrides).

Experiment

Read a running container's configuration from the outside, without entering it:

```
docker inspect dev --format '{{range .Config.Env}}{{println .}}{{end}}'
```

```
APP_ENV=development
GREETING=Hi from the dev container
PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
PYTHON_VERSION=3.14.8
PYTHON_SHA256=c2215904f02b175596dc49351585104f4bc20341e1c47378b26a2c274360ce73
```

...or from the inside:

```
docker exec dev printenv APP_ENV
```

Use an env file for many values:

```
printf 'APP_ENV=staging\nGREETING=From a file\n' > staging.env
docker run -d --name staging -p 5002:5000 --env-file staging.env simple-app:1.0
rm staging.env
```

```
curl -s http://localhost:5002
```

Note: in an env file, quotes are taken literally, so write `GREETING=From a file` without quotes.

Break It

Some images **require** a setting. PostgreSQL refuses to create a database without a password:

```
docker run --name pg postgres:18-alpine
```

Troubleshoot It

- **Observe:** the command returns immediately; the message starts with `Error: Database is uninitialized` and `superuser password is not specified`.
- **Investigate:**

```
docker ps -a --filter name=pg --format '{{.Names}}: {{.Status}}'
docker logs pg
```

- **Root cause:** the required variable `POSTGRES_PASSWORD` was not set. The image's documentation on Docker Hub lists the variables it understands.
- **Fix:** start a new container with the variable (options cannot be added to an existing container):

```
docker rm pg
docker run -d --name pg -e POSTGRES_PASSWORD=lab-only-password postgres:18-alpine
```

- **Verify:** after a few seconds, `docker ps` shows `pg` as **Up**, and the logs end with `database system is ready to accept connections`:

```
docker ps --filter name=pg --format '{{.Names}}: {{.Status}}'
docker logs pg 2>&1 | tail -3
```

The same pattern for our own API is scenario [06 · environment variable missing](#).

Common Mistakes

- Setting a variable with `-e` that the app never reads (check the app's docs or code).
- Forgetting that `-e` goes **before** the image name: `docker run -e A=1 image`, not `docker run image -e A=1` (after the image name it becomes an argument for the container's command).
- Expecting `docker restart` to pick up new values. Environment is fixed at creation; recreate the container.
- Putting real passwords in `-e`: anyone who can run `docker inspect` can read them (lesson [19 · Security Basics](#) shows secret files instead).
- Quoting values inside `--env-file`: the quotes become part of the value.
- (macOS) "port is already allocated" on 5000: the AirPlay Receiver uses port 5000. Use another host port (`-p 5050:5000`) or switch AirPlay Receiver off in System Settings.

Best Practices

- Give every setting a sensible default in the Dockerfile (`ENV APP_ENV=production`) or in code.
- Let the app **fail fast** with a clear message when a required setting is missing.
- Keep env files out of git when they contain secrets (`.gitignore`, `.dockerignore`).
- Use Docker secrets/secret files for passwords in anything shared or deployed.

Challenge

Run `simple-app:1.0` on host port 5005 so that it answers `Welcome to the challenge!` and `environment: challenge`, using exactly two `-e` options. Prove it with `curl` and with `docker inspect`.

Solution

▼ Show the solution

`app.py` adds the `!` itself, so the greeting is `Welcome to the challenge`:

```
docker run -d --name challenge -p 5005:5000 -e APP_ENV=challenge -e GREETING="Welcome to the challenge" simple-app:1.0
```

```
curl -s http://localhost:5005
```

```
docker inspect challenge --format '{{range .Config.Env}}{{println .}}{{end}}' | grep -E 'APP_ENV|GREETING'
```

Verification

- [] You can set variables with `-e` and `--env-file`.
- [] You know the order of precedence (Dockerfile `ENV` < env file < `-e`).
- [] You can read a container's environment with `docker inspect` and `docker exec printenv`.

- [] You can diagnose a container that exits because a variable is missing.

Clean up:

```
docker rm -f prod dev staging pg challenge
```

Real-World Usage

- Every 12-factor style service is configured through environment variables: database URLs, log levels, feature flags.
- The same image moves from test to production; only the variables change.
- Platforms (Compose, and later orchestrators) inject environment variables per environment.

Key Takeaways

- Environment variables configure a container at start time; the image stays the same.
- Precedence: Dockerfile `ENV` < `--env-file` < `-e`.
- Read them with `docker inspect` or `docker exec ... printenv`.
- Passwords in env vars are visible to anyone with Docker access: use secrets for real systems.
- Next: [09 · Dockerfile](#).

Dockerfile

Lesson 09 · about 60 minutes · you need: [08 · Environment Variables](#)

What is it?

A **Dockerfile** is a text file with step-by-step instructions to build an image. Each line is one instruction: start from this base image, copy these files, run this command, use this port, start this program. `docker build` reads it and produces an image.

```
Dockerfile  --(docker build)-->  Image  --(docker run)-->  Container
(recipe)                (package)                (running app)
```

Why do we need it?

Images like `nginx` or `python` are generic. Your application needs **its own** image: Python plus your code plus your dependencies. Writing that down as a Dockerfile makes the build:

- **repeatable:** anyone runs `docker build` and gets the same result;
- **reviewable:** it is a text file in git, changes are visible in pull requests;
- **automatable:** a build server can build it without a human clicking anything.

How does it work?

The instructions you need

Instruction	What it does	Example
FROM	the base image to start from (always first)	FROM python:3.14-slim
WORKDIR	the working directory for the next instructions (created if missing)	WORKDIR /app
COPY	copy files from the build context into the image	COPY app.py .
RUN	run a command while building (installs, setup); the result is saved in the image	RUN pip install -r requirements.txt
ENV	set default environment variables	ENV APP_ENV=production
EXPOSE	document the port the app listens on (does not publish it)	EXPOSE 5000
USER	which user runs the following instructions and the container	USER appuser
CMD	the default command when a container starts (easy to override)	CMD ["python", "app.py"]
ENTRYPOINT	the fixed program; <code>CMD</code> then supplies its default arguments	ENTRYPOINT ["echo", "Hello,"]

Other useful ones you will meet later: `ARG` (build-time variable), `LABEL` (metadata), `HEALTHCHECK` (how Docker checks the app is healthy), `COPY --from=` (multi-stage builds, [lesson 20 · Image Optimization](#)).

RUN versus CMD

```
RUN  pip install flask    -> happens ONCE, during docker build, result stored in the image
CMD  python app.py        -> happens EVERY time a container starts, during docker run
```

ENTRYPOINT versus CMD

```
ENTRYPOINT ["echo", "Hello,"]    CMD ["world"]

docker run greeter                -> echo Hello, world
docker run greeter Docker         -> echo Hello, Docker    (arguments replace CMD only)
```

The build context

`docker build -t name .` — the final `.` is the **build context**: the folder whose files Docker may `COPY`. Docker sends that folder (minus what `.dockerignore` excludes) to the engine before building. `COPY` can only see files inside it.

Exec form versus shell form

Write `CMD ["python", "app.py"]` (JSON "exec form"), not `CMD python app.py` ("shell form"). The exec form starts Python directly as PID 1, so it receives the stop signal from `docker stop` and shuts down cleanly.

Prerequisites

- `examples/simple-app/` from this repository: `app.py` (a 30-line Flask app), `requirements.txt` (`flask==3.1.3`) and the step files in `dockerfile-steps/`.

Hands-on Lab

We build the image in steps, adding only what the previous step proved we need. Go to the app:

```
cd examples/simple-app
```

Step 1 · only a base image. `dockerfile-steps/01.Dockerfile` contains just `FROM python:3.14-slim`.

```
docker build -f dockerfile-steps/01.Dockerfile -t simple-app:step1 .
```

It runs Python, but there is no app in it yet:

```
docker run --rm simple-app:step1 python --version
```

Step 2 · add the code and a start command (`WORKDIR /app`, `COPY . .`, `CMD ["python", "app.py"]`):

```
docker build -f dockerfile-steps/02.Dockerfile -t simple-app:step2 .
```

```
docker run --rm simple-app:step2
```

The image builds, but the app crashes: `ModuleNotFoundError: No module named 'flask'`. The base image has Python, not our dependency. We need `RUN`.

Step 3 · install the dependency (`RUN pip install --no-cache-dir -r requirements.txt`):

```
docker build -f dockerfile-steps/03.Dockerfile -t simple-app:step3 .
docker run -d --name step3 -p 5000:5000 simple-app:step3
```

```
curl -s http://localhost:5000
```

Step 4 · configuration and documentation (`ENV APP_ENV=production` , `EXPOSE 5000`):

```
docker build -q -f dockerfile-steps/04.Dockerfile -t simple-app:step4 .
docker image inspect simple-app:step4 --format 'env: {{.Config.Env}}
ports: {{.Config.ExposedPorts}}
cmd: {{.Config.Cmd}}'
```

```
sha256:753c5e1653cd669655f850cc43f34564d3224b666fdfaa4454757476c13e7128
env: [PATH=/usr/local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
PYTHON_VERSION=3.14.8 PYTHON_SHA256=c2215904f02b175596dc49351585104f4bc20341e1c47378b26a2c274360ce73
APP_ENV=production]
ports: map[5000/tcp:{}]
cmd: [python app.py]
```

Steps 5 (cache-friendly order) and 6 (non-root user) follow in lessons [10 · Image Layers](#) and [19 · Security Basics](#). The finished result is `examples/simple-app/Dockerfile`. The full guided lab is [labs/06-first-dockerfile](#).

Expected Result

- Each `docker build` ends with a line that names the image (naming to `...simple-app:stepN`).
- Step 1 prints `Python 3.14.x` ; step 2 fails with `No module named 'flask'` ; step 3 answers `Hello from simple-app!` .
- The step 4 image carries `APP_ENV=production` and documents `5000/tcp` .
- `docker images simple-app` lists one image per tag.

Experiment

ENTRYPOINT and CMD together, with `dockerfile-steps/entrypoint-vs-cmd.Dockerfile` :

```
docker build -q -f dockerfile-steps/entrypoint-vs-cmd.Dockerfile -t greeter .
docker run --rm greeter
docker run --rm greeter Docker
```

Override the CMD of our app at run time: the image stays the same, only this container runs something else:

```
docker run --rm simple-app:step3 pip show flask
```

Break It

Build a Dockerfile with a typo in an instruction (here we pipe a two-line Dockerfile straight into `docker build` with `-`; such a build has no context, which is fine without `COPY`):

```
printf 'FROM python:3.14-slim\nRUNN echo hello\n' | docker build -t typo -
```

Troubleshoot It

- **Observe:** the build stops before doing anything: `dockerfile parse error ... unknown instruction: RUNN` (BuildKit even suggests `did you mean RUN?`).
- **Investigate:** the error names the line number and the word it did not understand.
- **Root cause:** `RUNN` is not an instruction.
- **Fix and verify:**

```
printf 'FROM python:3.14-slim\nRUN echo hello > /greeting.txt\nCMD ["cat", "/greeting.txt"]\n' | docker build -q -t typo -\ndocker run --rm typo
```

A different build failure, a `COPY` of a file that is not in the build context, is scenario [05 · Dockerfile build fails](#).

Common Mistakes

- `COPY . .` before installing dependencies: every code change re-installs everything (lesson [10 · Image Layers](#)).
- Using `RUN python app.py` to "start" the app: `RUN` executes at build time and the build then hangs (a server never finishes). Starting is `CMD`'s job.
- Shell-form `CMD python app.py`: signals go to a shell, not your app; shutdowns become slow.
- `FROM python` without a tag: you silently get whatever `latest` is today.
- Copying secrets (`.env`, keys) into the image because `.dockerignore` is missing (lesson [11 · .dockerignore](#)).

Best Practices

- Pin the base image tag (`python:3.14-slim`).
- Order instructions from "rarely changes" to "often changes": base, system packages, dependencies, then your code.
- Use exec-form `CMD / ENTRYPOINT` .
- Add a `.dockerignore` , and run as a non-root `USER` .
- Keep one Dockerfile per image, next to the code it builds.

Challenge

Write your own Dockerfile (call it `my.Dockerfile` , in `examples/simple-app/`) that builds a working simple-app image whose **default greeting** is `Built by me` , without changing `app.py` . Run it on port 5050 and prove it with curl.

Solution

▼ Show the solution

The greeting comes from the `GREETING` environment variable, so an `ENV` default is all it takes:

```
cat > my.Dockerfile <<'EOF'
FROM python:3.14-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
ENV GREETING="Built by me"
EXPOSE 5000
CMD ["python", "app.py"]
EOF
docker build -q -f my.Dockerfile -t simple-app:mine .
docker rm -f mine 2>/dev/null; docker run -d --name mine -p 5050:5000 simple-app:mine
sleep 2
curl -s http://localhost:5050
```

Verification

- [] You can explain what each of FROM, WORKDIR, COPY, RUN, ENV, EXPOSE, CMD, ENTRYPOINT does.
- [] You know the difference between build time (`RUN`) and run time (`CMD`).
- [] You can build with `-t` (name) and `-f` (which Dockerfile).
- [] You understand what the build context is.

Clean up (and go back to the repository root):

```
docker rm -f step3 mine
rm -f my.Dockerfile
cd ../../
```

Real-World Usage

- Every service a team owns has a Dockerfile in its repository; build servers build it on every change.
- Code review of a Dockerfile catches problems early: unpinned bases, root users, secrets copied in.
- `ENTRYPOINT` + `CMD` is the standard pattern for CLI tools packaged as images.

Key Takeaways

- A Dockerfile is a reproducible recipe for an image.
- `RUN` = build time, `CMD` / `ENTRYPOINT` = run time.
- The build context is the folder you pass to `docker build`; `COPY` sees only that.
- Build up a Dockerfile in small, tested steps.
- Next: [10 · Image Layers](#).

Image Layers

Lesson 10 · about 40 minutes · you need: 09 · [Dockerfile](#)

What is it?

An image is not one big file. It is a **stack of read-only layers**. Every instruction in a Dockerfile that changes files (`FROM`, `COPY`, `RUN`) adds one layer on top of the previous ones. Instructions that only change settings (`ENV`, `EXPOSE`, `CMD`, `USER`) add metadata, with a layer of size 0.

Dockerfile (step 5)		Image layers (bottom to top)
FROM python:3.14-slim	-->	[debian slim + python] ~120 MB
WORKDIR /app	-->	[empty /app folder] 0 B
COPY requirements.txt .	-->	[requirements.txt] tiny
RUN pip install --no-cache-dir -r requirements.txt	-->	[flask + dependencies] a few MB
COPY app.py .	-->	[app.py] tiny
ENV / EXPOSE / CMD	-->	(metadata only, 0 B)

container = all of the above (read-only) + [its own writable layer]

Why do we need it?

Layers make Docker fast and economical:

- **Sharing:** ten images built `FROM python:3.14-slim` store the Python layers **once** on disk.
- **Caching:** when you rebuild, Docker reuses every layer whose inputs did not change. A rebuild after a one-line code change can take one second instead of a minute.
- **Transfer:** pushing and pulling only moves layers the other side does not already have.

How does it work?

The build cache rule

For each instruction, Docker asks: "Have I already built **this exact instruction** on top of **this exact parent layer**, with **these exact files**?" If yes, it reuses the layer (`CACHED`). If no, it rebuilds this layer **and every layer after it**.

Bad order (step 3)		Good order (step 5)	
FROM python:3.14-slim	cached	FROM python:3.14-slim	cached
WORKDIR /app	cached	WORKDIR /app	cached
COPY . .	CHANGED (app.py)	COPY requirements.txt .	cached
RUN pip install ...	REBUILT (slow)	RUN pip install ...	cached (fast!)
		COPY app.py .	CHANGED -> rebuilt (tiny)

`COPY` decides "changed" by looking at the **content** of the copied files. So `COPY . .` breaks the cache whenever **any** file in the context changes, and everything after it runs again.

Order rule of thumb

Put what changes **rarely** at the top and what changes **often** at the bottom: base image → system packages → dependency list → install dependencies → your code.

Prerequisites

- `examples/simple-app/` and the step files 03 and 05 from lesson 09.

Hands-on Lab

```
cd examples/simple-app
```

Build step 5 and look at its layers with `docker history` (newest layer at the top):

```
docker build -q -f dockerfile-steps/05.Dockerfile -t simple-app:step5 .
docker history simple-app:step5 --format 'table {{.CreatedBy}}\t{{.Size}}' | head -12
```

```
sha256:bc05088365fac70886ec410303b7ecf91170caf17f542bd0e8c9b8fab30809c
CREATED BY                                SIZE
CMD ["python" "app.py"]                   0B
EXPOSE [5000/tcp]                         0B
ENV APP_ENV=production                    0B
COPY app.py . # buildkit                  12.3kB
RUN /bin/sh -c pip install --no-cache-dir -r... 16.9MB
COPY requirements.txt . # buildkit         12.3kB
WORKDIR /app                             8.19kB
CMD ["python3"]                           0B
RUN /bin/sh -c set -eux; for src in idle3 p... 16.4kB
RUN /bin/sh -c set -eux; savedAptMark="$(a... 42MB
ENV PYTHON_SHA256=c2215904f02b175596dc493515... 0B
```

Build it **again** without changing anything. Every step is `CACHED`:

```
docker build --progress=plain -f dockerfile-steps/05.Dockerfile -t simple-app:step5 . 2>&1 | grep -E '^#
[0-9]+ (\[|CACHED)'
```

```
#1 [internal] load build definition from 05.Dockerfile
#2 [internal] load metadata for docker.io/library/python:3.14-slim
#3 [internal] load .dockerignore
#4 [internal] load build context
#5 [1/5] FROM docker.io/library/python:3.14-slim@sha256:c3e521df8b2b498a7a682e7e18676771cb80c6b75b8699af886b2d554ce40151
#6 [3/5] COPY requirements.txt .
#6 CACHED
#7 [2/5] WORKDIR /app
#7 CACHED
#8 [4/5] RUN pip install --no-cache-dir -r requirements.txt
#8 CACHED
#9 [5/5] COPY app.py .
#9 CACHED
```

The full guided lab, with timings, is [labs/07-layers-and-cache](#).

Expected Result

- `docker history` lists one row per instruction. The `RUN pip install` row has a size of a few MB, `COPY app.py` a few kB, the `ENV / EXPOSE / CMD` rows `0B`. Rows from the base image appear below ours.
- The repeated build shows `CACHED` for every step and finishes in about a second.

Experiment

Now change the code and compare the two Dockerfile orders. Keep a backup of `app.py` first:

```
cp app.py app.py.bak
echo "# a small code change $(date +%s)" >> app.py
```

Step 3 order (`COPY . .` before `pip install`): the code change forces pip to run again. `Collecting flask` in the output proves it:

```
docker build --progress=plain -f dockerfile-steps/03.Dockerfile -t simple-app:step3 . 2>&1 | grep -E
'CACHE|Collecting flask|RUN pip'
```

Change the code once more and rebuild with the **step 5 order**: pip is `CACHED`, only `COPY app.py` runs:

```
echo "# another change $(date +%s)" >> app.py
docker build --progress=plain -f dockerfile-steps/05.Dockerfile -t simple-app:step5 . 2>&1 | grep -E
'CACHE|Collecting flask|COPY app.py'
```

Restore the original file:

```
mv app.py.bak app.py
```

Break It

A layer cannot be undone by a later layer. Watch what happens when we create a big file in one `RUN` and delete it in the next:

```
printf 'FROM alpine:3.24\nRUN head -c 50000000 /dev/zero > /big.file\nRUN rm /big.file\n' | docker build
-q -t layers-trap -
docker images layers-trap --format 'layers-trap: {{.Size}}'
```

Troubleshoot It

- **Observe:** the image is about 50 MB bigger than `alpine:3.24`, although `/big.file` does not exist in it.
- **Investigate:** `docker history` shows where the size comes from:

```
docker history layers-trap --format 'table {{.CreatedBy}}\t{{.Size}}' | head -4
```

CREATED BY	SIZE
RUN /bin/sh -c rm /big.file # buildkit	4.1kB
RUN /bin/sh -c head -c 50000000 /dev/zero > ...	50MB
CMD ["/bin/sh"]	0B

- **Root cause:** the `head -c ...` layer contains the 50 MB file forever. The `rm` layer only adds a marker "this file is deleted" on top; the bytes are still shipped.
- **Fix:** create and delete in the **same** `RUN` (or never create it), so no layer ever contains it.
- **Verify:**

```
printf 'FROM alpine:3.24\nRUN head -c 50000000 /dev/zero > /big.file && rm /big.file\n' | docker build -q -t layers-fixed -
docker images --format 'table {{.Repository}}\t{{.Size}}' | grep -E 'REPOSITORY|layers-'
```

```
sha256:98c74ac70f902fa1225d8408c928da912ce3abeeeea3cf120d8f80a4cfbd6e83f
REPOSITORY      SIZE
layers-fixed    12.9MB
layers-trap     63MB
```

This is exactly why `apt-get update && apt-get install ... && rm -rf /var/lib/apt/lists/*` is written as **one** `RUN`.

Common Mistakes

- `COPY . .` at the top of the Dockerfile: every change reinstalls everything.
- Deleting files in a later `RUN` to "make the image smaller": it does not.
- Splitting `apt-get update` and `apt-get install` into two `RUN`s: the cached `update` layer becomes stale, and installs fail or install old versions.
- Putting secrets into a layer and deleting them later: they stay readable in the earlier layer.

Best Practices

- Order instructions from least to most frequently changed.
- Copy the dependency list alone first (`requirements.txt` , `package.json` , `go.mod`), install, then copy the code.
- Combine commands that create and clean up temporary files into one `RUN`.
- Use `docker history` to find which instruction made an image big.

Challenge

Using `docker history`, find out **which instruction** of `simple-app:step5` adds the most size **on top of the base image**, and how big it is.

Solution

▼ Show the solution

`--no-trunc` shows the full instruction; sort by size isn't built in, so read the table:

```
docker history simple-app:step5 --format '{{.Size}}\t{{.CreatedBy}}' | head -6
```

The `RUN pip install --no-cache-dir -r requirements.txt` row is the biggest of our own layers (Flask and its dependencies). Everything below the `WORKDIR /app` row belongs to `python:3.14-slim`.

Verification

- [] You can explain what a layer is and which instructions create one.
- [] You can predict which steps rebuild after a change.
- [] You can read `docker history`.
- [] You know why deleting a file in a later layer does not shrink the image.

Clean up (and return to the repository root):

```
docker rmi layers-trap layers-fixed  
cd ../../
```

Real-World Usage

- Build servers rebuild images many times a day; a cache-friendly Dockerfile saves minutes per build and a lot of bandwidth.
- When an image suddenly grows by hundreds of MB, `docker history` points to the instruction.
- Shared base images across a company mean most layers are already present on every server.

Key Takeaways

- Images are stacks of read-only layers; containers add one writable layer.
- A changed layer invalidates every layer after it.
- Dependencies first, code last.
- Deleting in a later layer never removes bytes from an earlier one.
- Next: [11 · .dockerignore](#).

.dockerignore

Lesson 11 · about 30 minutes · you need: 10 · [Image Layers](#)

What is it?

`.dockerignore` is a file next to your Dockerfile that lists files and folders Docker must **not** send to the build. It uses the same idea as `.gitignore`.

```
examples/simple-app/          sent to the engine (build context)
├─ app.py                     -----> app.py
├─ requirements.txt           -----> requirements.txt
├─ .env                       (secrets!)   X   (ignored)
├─ junk/                      (60 MB)      X   (ignored)
├─ dockerfile-steps/         X             (ignored)
└─ .dockerignore              (read by docker build, not sent)
```

Why do we need it?

Before the first instruction runs, `docker build` packs up the **whole build context** folder and sends it to the engine. Without `.dockerignore`:

- **builds are slow:** large folders (`.git`, `node_modules`, test data, logs) are transferred every time;
- **secrets leak:** `COPY . .` copies your `.env`, SSH keys or credentials **into the image**, where anyone who gets the image can read them;
- **the cache breaks more often:** an unrelated file changing (a log file) invalidates `COPY . .`.

How does it work?

`docker build -t name .` → the CLI reads `./.dockerignore`, leaves out every match, and sends the rest.
Patterns:

Pattern	Matches
<code>.env</code>	the file <code>.env</code> in the context root
<code>*.log</code>	every <code>.log</code> file in the root
<code>**/*.log</code>	<code>.log</code> files in any folder
<code>junk/</code>	the folder <code>junk</code> and everything in it
<code>!keep.log</code>	exception: do send <code>keep.log</code> after all

This repository's `examples/simple-app/.dockerignore`:

```
.git
.env
*.log
__pycache__/
*.pyc
junk/
dockerfile-steps/
optimization/
Dockerfile*
.dockerignore
```

`docker build -f dockerfile-steps/03.Dockerfile .` still works although `dockerfile-steps/` is ignored: the CLI always sends the Dockerfile you name with `-f` separately.

Prerequisites

- `examples/simple-app/` and the step 02 Dockerfile (`COPY . .`) from lesson 09.

Hands-on Lab

We create the two kinds of files that cause trouble: a big folder and a fake secret.

```
cd examples/simple-app
mkdir -p junk
head -c 60000000 /dev/zero > junk/big-test-data.bin
echo "DB_PASSWORD=super-secret-do-not-ship" > .env
```

Before: switch the `.dockerignore` off by renaming it, and build with `COPY . .`:

```
mv .dockerignore dockerignore.off
docker build --progress=plain -f dockerfile-steps/02.Dockerfile -t ignore-test:before . 2>&1 | grep
'transferring context'
```

```
#3 transferring context: 2B done
#5 transferring context: 60.02MB 2.2s done
```

Look inside the image:

```
docker run --rm ignore-test:before ls -a /app
docker run --rm ignore-test:before cat /app/.env
```

```
.
..
.env
Dockerfile
app.py
dockerfile-steps
dockerignore.off
junk
optimization
requirements.txt
DB_PASSWORD=super-secret-do-not-ship
```

After: switch the `.dockerignore` back on and build again:

```
mv dockerignore.off .dockerignore
docker build --progress=plain -f dockerfile-steps/02.Dockerfile -t ignore-test:after . 2>&1 | grep
'transferring context'
```

```
#3 transferring context: 295B done
#4 transferring context: 63B 0.0s done
```

```
docker run --rm ignore-test:after ls -a /app
```

```
.
..
app.py
requirements.txt
```

The guided lab is [labs/08-dockerignore](#).

Expected Result

- **Before:** `transferring context` shows tens of MB; the image contains `.env` (with the "password") and `junk/`.
- **After:** the context is a few kB; `/app` contains only the app files.

Compare the image sizes too:

```
docker images ignore-test --format 'table {{.Tag}}\t{{.Size}}'
```

TAG	SIZE
before	250MB
after	189MB

Experiment

`docker history` shows that the 60 MB came in with the `COPY . .` layer:

```
docker history ignore-test:before --format '{{.Size}}\t{{.CreatedBy}}' | head -3
```

Break It

Ignore rules can also be **too greedy**. Scenario [05 · Dockerfile build fails](#) has a `.dockerignore` with `*.txt` in it. Build it:

```
docker build -t ts05 ../../troubleshooting/05-dockerfile-build-fails
```

Troubleshoot It

- **Observe:** the build fails at `COPY requirements.txt .` with `failed to compute cache key: ... "/requirements.txt": not found`.
- **Investigate:** the file **does** exist on disk (`ls ../../troubleshooting/05-dockerfile-build-fails`). "Not found" means "not in the build context". Read that folder's `.dockerignore`:

```
cat ../../troubleshooting/05-dockerfile-build-fails/.dockerignore
```

- **Root cause:** `*.txt` excludes `requirements.txt` from the context.
- **Fix and verify:** the scenario's README walks through the fix. In short: be specific (`notes.txt`, or `*.log` only), or add an exception line `!requirements.txt`.

Common Mistakes

- No `.dockerignore` at all, together with `COPY . .`.
- Putting `.dockerignore` in the wrong folder: it must be in the **root of the build context** (the folder you pass to `docker build`), not next to a Dockerfile in a subfolder.
- Believing `.env` is safe because it is in `.gitignore`. Git and Docker read different ignore files.
- Patterns that are too broad (`*.txt`, `*.json`) and exclude files the build needs.

Best Practices

- Create `.dockerignore` together with the Dockerfile, always.
- Start with: `.git`, `.env`, `*.log`, dependency folders (`node_modules`, `__pycache__`), test data.
- Prefer copying only what you need (`COPY app.py .`) over `COPY . .`.
- Never rely on `.dockerignore` alone for secrets: keep secrets out of the project folder, or pass them at run time (lesson 19 · [Security Basics](#)).

Challenge

Without renaming any file, prove that `.env` and `junk/` are **not** sent with the current `.dockerignore`, using a throw-away Dockerfile piped into `docker build`. Hint: a Dockerfile can `COPY . /ctx` and list `/ctx`.

Solution

▼ Show the solution

`-f -` reads the Dockerfile from standard input while `.` is still the build context:

```
printf 'FROM alpine:3.24\nCOPY . /ctx\nCMD ["ls", "-a", "/ctx"]\n' | docker build -q -t context-check -f - .
docker run --rm context-check
```

Verification

- [] You can explain what the build context is and when it is sent.
- [] You can write `.dockerignore` patterns.

- [] You proved that a missing `.dockerignore` puts secrets into an image.
- [] You can recognise a build failure caused by an over-eager ignore rule.

Clean up (delete the test data and return to the repository root):

```
rm -rf junk .env
docker rmi ignore-test:before ignore-test:after context-check
cd ../../
```

Real-World Usage

- Leaked credentials inside public images are a real, recurring security incident; a `.dockerignore` is the cheapest protection.
- Large monorepos can have gigabytes in the build context; ignoring them turns minute-long "sending context" phases into seconds.
- Security reviews check `.dockerignore` next to every Dockerfile.

Key Takeaways

- The build context is sent before the build; `.dockerignore` decides what is in it.
- Without it, builds are slower and secrets can end up inside images.
- "not found" in `COPY` usually means "excluded from or outside the context".
- Next: [12 · Volumes](#).

Volumes

Lesson 12 · about 45 minutes · you need: [04 · Containers](#), [06 · Container Lifecycle](#)

What is it?

A **volume** is storage that Docker manages **outside** any container's writable layer. You mount it into a container at a path (for example `/data`). Files written there survive when the container is removed, and another container can mount the same volume later.

```
docker run -v notes:/data ...
```



Why do we need it?

A container's writable layer is deleted with the container. That is great for apps and terrible for **data**. Databases, uploaded files and anything you must keep belong on a volume, so you can upgrade, recreate or replace the container without losing them.

How does it work?

Command	What it does
<code>docker volume create notes</code>	create a named volume
<code>docker volume ls</code>	list volumes
<code>docker volume inspect notes</code>	details: driver, Mountpoint (where the data really is), labels
<code>docker volume rm notes</code>	delete it (refused while a container uses it)
<code>docker run -v notes:/data ...</code>	mount volume <code>notes</code> at <code>/data</code> (created automatically if missing)
<code>docker run --mount type=volume,src=notes,dst=/data ...</code>	the same, longer but more explicit

Three kinds of storage you will meet:

named volume	-v notes:/data	managed by Docker, has a name	(this lesson)
anonymous volume	-v /data or image VOLUME	managed by Docker, random name	(easy to lose!)
bind mount	-v "\$(pwd)/site:/data"	a folder of YOUR computer	(lesson 13)

If the volume is new and empty, Docker first copies whatever the image already has at that path into it. If it already contains data, the image's files at that path are hidden by the volume.

Prerequisites

- `alpine:3.24` and `postgres:18-alpine` images (pulled automatically).

Hands-on Lab

Create a volume and look at it:

```
docker volume create notes
docker volume ls
docker volume inspect notes
```

```
notes
DRIVER      VOLUME NAME
local      notes
[
  {
    "CreatedAt": "2026-10-04T05:04:06Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/notes/_data",
    "Name": "notes",
    "Options": null,
    "Scope": "local"
  }
]
```

Write data with one container, then **remove that container**:

```
docker run --name writer -v notes:/data alpine:3.24 sh -c 'echo "written by the writer at $(date)" > /data/note.txt; cat /data/note.txt'
docker rm writer
```

A brand-new container mounts the same volume and finds the file:

```
docker run --rm --name reader -v notes:/data alpine:3.24 cat /data/note.txt
```

The guided lab, with more experiments, is [labs/09-volumes](#).

Expected Result

- `docker volume inspect` shows `"Driver": "local"` and a `"Mountpoint"` such as `/var/lib/docker/volumes/notes/_data` (on Docker Desktop that path is inside its Linux VM).
- The reader prints `written by the writer at ...`: the data outlived the container that wrote it.

Experiment

Compare with **no** volume. Data written to the container's own layer dies with the container:

```
docker run --name temp alpine:3.24 sh -c 'echo "I will be lost" > /tmp/note.txt'
docker rm temp
docker run --rm alpine:3.24 cat /tmp/note.txt 2>&1 || true
```

A real database. PostgreSQL 18 keeps its data under `/var/lib/postgresql`, so we mount a volume there, create a table, delete the container, and start a new one on the same volume:

```
docker run -d --name db1 -e POSTGRES_PASSWORD=lab-only-password -v pgdata:/var/lib/postgresql
postgres:18-alpine
```

```
docker exec db1 psql -U postgres -c "CREATE TABLE IF NOT EXISTS notes (text TEXT); INSERT INTO notes
VALUES ('survives container removal');"
```

```
docker rm -f db1
docker run -d --name db2 -e POSTGRES_PASSWORD=lab-only-password -v pgdata:/var/lib/postgresql
postgres:18-alpine
```

Give PostgreSQL two or three seconds to start, then ask the **new** container:

```
docker exec db2 psql -U postgres -c "SELECT * FROM notes;"
```

If it says the server is still starting up, wait a moment and run it again. This time the database found existing data on the volume, so it skipped initialisation and just started.

Break It

Try to delete a volume that a container is using:

```
docker volume rm pgdata
```

Troubleshoot It

- **Observe:** Error response from daemon: remove pgdata: volume is in use - [`<container id>`].
- **Investigate:** which container? Filter containers by volume:

```
docker ps -a --filter volume=pgdata --format '{{.Names}} {{.Status}}'
```

- **Root cause:** Docker protects volumes that are mounted by a container (running **or** stopped).
- **Fix:** remove the container first, **then** the volume, and only if you really want to delete the data. There is no undo and no recycle bin.
- **Verify:**

```
docker rm -f db2
docker volume rm pgdata
```

```
docker volume ls
```

Related scenarios: [04 · volume data appears missing](#) (a typo in the volume name) and [10 · database data disappears](#) (no named volume, or the wrong path).

Common Mistakes

- A typo in the volume name (`-v note:/data` instead of `notes`): Docker silently creates a new, empty volume, and your data seems "gone".
- Mounting at the wrong path: the app writes somewhere else, the volume stays empty. PostgreSQL 18 needs `/var/lib/postgresql` ; older tutorials use `/var/lib/postgresql/data` , which the 18 image refuses.
- `docker rm -v` or `docker compose down -v` : the `-v` also deletes volumes.
- Relying on anonymous volumes: every new container gets a new one; old ones pile up unnoticed.

Best Practices

- Always use **named** volumes for data you care about.
- Know exactly which path the image stores data in (read the image's documentation).
- Back up volumes (for databases: with the database's own dump tool).
- Treat `docker volume rm` and `prune` as data deletion, because they are.

Challenge

Create a volume `counter` . Run a container three times, each time with `--rm` , that adds one line to `/data/visits.txt` on that volume. Then print how many lines the file has. It must be 3.

Solution

▼ Show the solution

```
docker volume create counter
for i in 1 2 3; do docker run --rm -v counter:/data alpine:3.24 sh -c 'echo visit >>
/data/visits.txt'; done
docker run --rm -v counter:/data alpine:3.24 wc -l /data/visits.txt
```

`--rm` deleted each container right away, yet every visit was kept: the file lives on the volume.

Verification

- [] You can create, list, inspect and remove volumes.
- [] You proved that data on a volume survives container removal.
- [] You know where PostgreSQL 18 keeps its data.
- [] You know why a typo in a volume name looks like data loss.

Clean up:

Real-World Usage

- Every containerised database (PostgreSQL, MySQL, Redis with persistence) runs on a volume.
- Upgrading a database container means: stop, start the new image version on the **same** volume.
- Volume backups are part of every serious operations runbook.

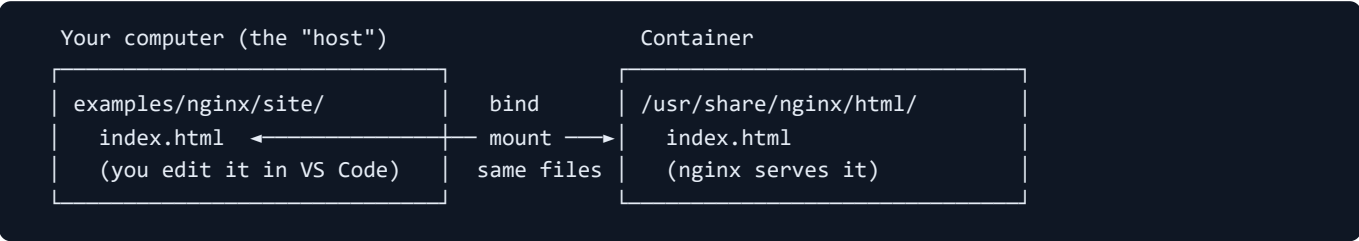
Key Takeaways

- Volumes keep data outside the container's lifecycle.
- Named volumes: `-v name:/path`; the name is the key, so typos create new empty volumes.
- Docker refuses to remove a volume in use; removing it deletes the data for good.
- Next: [13 · Bind Mounts](#).

Bind Mounts

What is it?

A **bind mount** connects a folder (or a single file) **on your computer** to a path **inside a container**. It is not a copy. Both sides look at the same files: change a file on your computer and the container sees the change immediately, and the other way round.



Why do we need it?

Rebuilding an image every time you change one line of HTML or code is slow. During **local development** you want to edit files with your normal editor and see the result in the running container at once. Bind mounts give you exactly that. They are also how you hand a container a single configuration file (the capstone mounts `database/init.sql` into the PostgreSQL container this way).

How does it work?

Docker asks the Linux kernel to make a host path visible at a container path. There are two ways to write it:

```
-v HOST_PATH:CONTAINER_PATH[:ro]          short, older syntax
--mount type=bind,src=HOST_PATH,dst=CONTAINER_PATH[,readonly] long, explicit syntax
```

	-v	--mount
Host path does not exist	Docker creates an empty folder for you (often hiding a typo)	Native Linux refuses: <code>bind</code> source path does not exist . Docker Desktop (Windows/Mac) may create it instead
Readability	short	every part is named
Read-only	<code>:ro</code> at the end	<code>,readonly</code>

Volume vs bind mount (see [12-volumes.md](#)):

	Named volume	Bind mount
Where the data lives	managed by Docker (<code>docker volume inspect</code>)	a folder you chose on your computer
Created with	<code>-v mydata:/path</code> (a name)	<code>-v "\$(pwd)/folder":/path</code> (a path)
Best for	database data, anything the app owns	source code, config files during development
Survives <code>docker rm</code>	yes	yes (it is your folder)

	Named volume	Bind mount
Portable between machines	yes	depends on your folder layout

The rule of thumb: **if the value before the `:` is a path, it is a bind mount; if it is a plain name, it is a volume.**

Windows users: use WSL2 (recommended) or PowerShell. In PowerShell write `${PWD}` instead of `$(pwd)`. In Git Bash run `export MSYS_NO_PATHCONV=1` first, otherwise Git Bash rewrites `/usr/share/...` into a Windows path.

Prerequisites

- [12-volumes.md](#) (what a volume is)
- [07-ports.md](#) (`-p 8080:80`)

Hands-on Lab

The full lab is [labs/10-bind-mounts](#). The short version:

```
cd examples/nginx
docker run -d --name web -p 8080:80 -v "$(pwd)/site":/usr/share/nginx/html:ro nginx:1.30-alpine
```

```
curl -s http://localhost:8080
```

nginx now serves the files from `examples/nginx/site/`, not the default nginx page. Let's add a new file **on the host** and ask the container for it, without restarting anything:

```
echo "<h1>Added on the host, seen by the container</h1>" > site/live.html
curl -s http://localhost:8080/live.html
```

Expected Result

- The first `curl` returns the page from `site/index.html` ("Hello from my container!").
- The second `curl` returns the file you just created on your computer. No rebuild, no restart.

Experiment

The mount is `:ro` (read-only) for the container. Your computer can still write, but the container cannot:

```
docker exec web sh -c "echo hacked > /usr/share/nginx/html/index.html"
```

That is a good default for anything a container only needs to **read**.

Clean up the experiment file:

```
rm site/live.html
docker rm -f web
```

Break It

A typo in the host path is one of the most common real mistakes. With `-v`, Docker silently creates the missing folder, so the container starts happily and serves... an empty folder:

```
docker run -d --name web -p 8080:80 -v "$(pwd)/sitee":/usr/share/nginx/html nginx:1.30-alpine
```

```
curl -s http://localhost:8080
```

nginx answers **403 Forbidden**: there is no `index.html` in an empty folder.

Troubleshoot It

Observe → investigate → root cause → fix → verify.

```
docker inspect web --format '{{json .Mounts}}'
```

The `Source` shows `.../sitee`: a folder you never meant to use. Check the host:

```
ls -d sitee
```

Root cause: a typo in the host path, and `-v` created the folder instead of complaining. Fix it, then verify:

```
docker rm -f web
rmdir sitee
docker run -d --name web -p 8080:80 -v "$(pwd)/site":/usr/share/nginx/html:ro nginx:1.30-alpine
```

```
curl -s http://localhost:8080
```

Use `--mount` when you want Docker to **refuse** a missing path on Linux (Docker Desktop may still create it). The full investigation is in [troubleshooting/09-host-changes-not-reflected](#).

```
docker rm -f web
```

Common Mistakes

- Relative paths: `-v site:/usr/share/nginx/html` is a **volume named "site"**, not your folder. Use `"$(pwd)/site"`.
- Forgetting the quotes around `"$(pwd)/..."` when the path contains spaces.
- Mounting over a folder that already has files in the image: the image's files are **hidden** while the mount exists.
- Editing files on the host for a container that was started from an image built with `COPY` and no mount: nothing changes until you rebuild.
- Git Bash on Windows rewriting container paths (use WSL2, PowerShell, or `MSYS_NO_PATHCONV=1`).

Best Practices

- Bind mounts for development and for single config files; named volumes for data the application owns.
- Add `:ro` whenever the container only reads.
- Prefer `--mount` in scripts so a wrong path fails loudly (on Linux).
- Never bind-mount your whole home folder or `/` into a container.

Challenge

Task: serve the page from `examples/nginx/site` on port 8082, read-only, using the `--mount` syntax instead of `-v`.

Requirements: container name `web2`; `curl http://localhost:8082` shows "Hello from my container".

Hints: `--mount type=bind,src=...,dst=...,readonly`.

Solution

▼ Solution

Still inside `examples/nginx`:

```
docker run -d --name web2 -p 8082:80 --mount
type=bind,src="$(pwd)/site",dst=/usr/share/nginx/html,readonly nginx:1.30-alpine
```

```
curl -s http://localhost:8082
docker rm -f web2
```

`--mount` names every part (`type` , `src` , `dst` , `readonly`), which makes the command longer but much easier to read.

Verification

- [] I can explain the difference between a bind mount and a named volume.
- [] I can start nginx with my own folder mounted and see live changes.
- [] I know why a typo in a `-v` path gives an empty folder (and a 403 from nginx).
- [] I know how to make a mount read-only.

Real-World Usage

DevOps engineers use bind mounts for **local development environments** (code mounted into a container with the right language version), for **configuration files** (nginx configs, database init scripts) and for **sharing build output** between the host and a container. Production data usually lives on named volumes or managed storage instead.

Key Takeaways

- A bind mount is a live window from the container into a folder on your computer.
- Path before the `:` → bind mount; plain name → volume.

- `-v` creates missing folders silently; `--mount` (on Linux) refuses them.
- Use `:ro` when the container only needs to read.

Docker Networking

What is it?

Every container gets its own network stack: its own IP address, its own `localhost`, its own ports. A **Docker network** is a virtual switch that containers plug into. Containers on the **same** network can talk to each other; containers on **different** networks cannot.

Why do we need it?

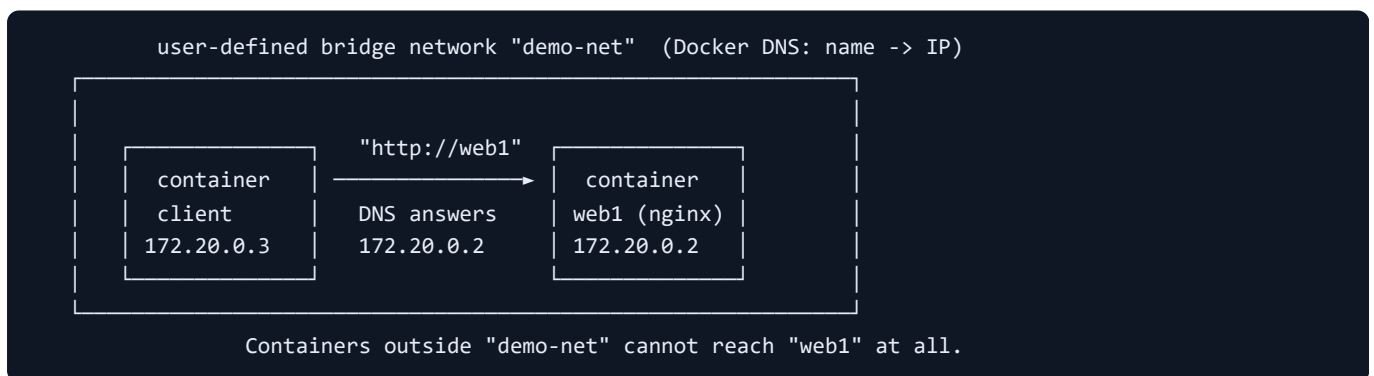
Real applications are several programs: a web server talks to an API, the API talks to a database. Each runs in its own container, so they need a safe, predictable way to find each other. Docker networks give you:

- **Communication** between containers, without publishing ports to your computer.
- **Names instead of IP addresses**: the API connects to `db`, not to `172.18.0.3` (IPs change every restart).
- **Isolation**: the web container does not even need to see the database.

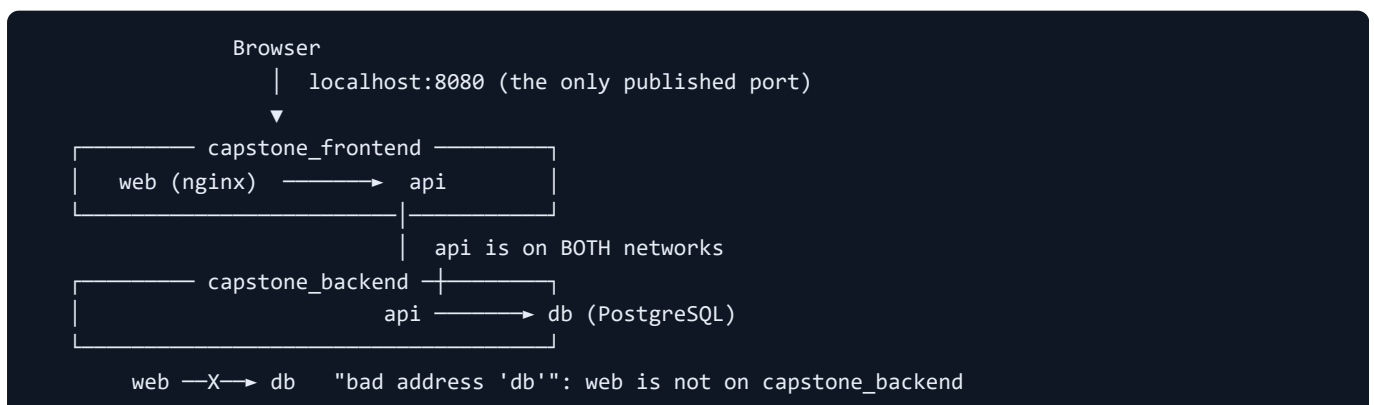
How does it work?

When you install Docker you get a network called `bridge` (the default). Containers started without `--network` join it. On the default bridge, containers get IP addresses but **no name resolution**.

When you create your **own** network (`docker network create`), Docker also runs a small **DNS server** for it: every container on that network can be reached by its **container name** (and in Compose, by its **service name**).



The capstone uses **two** networks so the web server can never touch the database:



Useful commands:

<code>docker network ls</code>	list networks
<code>docker network create NAME</code>	create a user-defined bridge network
<code>docker network inspect NAME</code>	which containers are attached, their IPs, the subnet
<code>docker network connect NAME CONTAINER</code>	plug a running container into another network
<code>docker network disconnect NAME CONTAINER</code>	unplug it
<code>docker network rm NAME</code>	remove a network (no containers may be attached)

Prerequisites

- [04-containers.md](#), [07-ports.md](#)

Hands-on Lab

Full lab: [labs/11-networking](#). The short version: create a network, start nginx on it, and reach it **by name** from a second container.

```
docker network create demo-net
docker run -d --name web1 --network demo-net nginx:1.30-alpine
```

```
docker run --rm --network demo-net busybox:1.37 wget -qO- http://web1
```

```
docker network inspect demo-net --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}{{"\n"}}{{end}}'
```

```
web1 172.18.0.2/16
```

Expected Result

- `wget` prints the HTML of the nginx welcome page: the busybox container found `web1` by its name.
- `docker network inspect` lists `web1` with an IP address in the network's subnet. (The busybox container is already gone, because `--rm` removed it when `wget` finished.)

Experiment

Do the same on the **default** bridge network, without `--network` :

```
docker run -d --name web2 nginx:1.30-alpine
```

```
docker run --rm busybox:1.37 wget -qO- -T 3 http://web2
```

`bad address 'web2'` : the default bridge has **no DNS for container names**. This is why you should always create your own network (or let Compose create one).

Now plug `web2` into `demo-net` while it is running, and try again from `demo-net` :

```
docker network connect demo-net web2
```

```
docker run --rm --network demo-net busybox:1.37 wget -q0- -T 3 http://web2
```

web2 is now on two networks at the same time, exactly like the capstone's api container.

Break It

Unplug web1 from demo-net and try to reach it:

```
docker network disconnect demo-net web1
```

```
docker run --rm --network demo-net busybox:1.37 wget -q0- -T 3 http://web1
```

Troubleshoot It

When two containers cannot talk, do not guess. Check **which networks each one is on**:

```
docker inspect web1 --format '{{range $name, $net := .NetworkSettings.Networks}}{{$name}} {{end}}'
docker inspect web2 --format '{{range $name, $net := .NetworkSettings.Networks}}{{$name}} {{end}}'
```

web1 is only on bridge; web2 is on bridge and demo-net. They do not share a user-defined network, so there is no name resolution between them. Fix and verify:

```
docker network connect demo-net web1
```

```
docker run --rm --network demo-net busybox:1.37 wget -q0- -T 3 http://web1
```

The Compose version of this problem (host not found in upstream "api") is in [troubleshooting/03-containers-cannot-communicate](#).

```
docker rm -f web1 web2
docker network rm demo-net
```

Common Mistakes

- Using container IP addresses in configuration. They change; use names.
- Expecting names to work on the default bridge network.
- Using localhost to reach **another** container. Inside a container, localhost is the container itself.
- Publishing database ports (-p 5432:5432) just so another container can reach it. Containers on the same network do not need published ports at all.
- Removing a network that still has containers attached (Docker refuses; stop or disconnect them first).

Best Practices

- One user-defined network per application (Compose does this automatically).
- Split networks by trust: the database only on a backend network.
- Publish only the ports humans need (usually just the web port).

- Refer to services by name everywhere (`DB_HOST=db`).

Challenge

Task: create a network `shop-net` , run nginx named `shop` on it, and prove that a busybox container on `shop-net` can reach it while a busybox container on the default bridge cannot.

Hints: `docker network create` , `--network` , `wget -q0- -T 3` .

Solution

▼ Solution

```
docker network create shop-net
docker run -d --name shop --network shop-net nginx:1.30-alpine
```

```
docker run --rm --network shop-net busybox:1.37 wget -q0- -T 3 http://shop
```

```
docker run --rm busybox:1.37 wget -q0- -T 3 http://shop
```

```
docker rm -f shop
docker network rm shop-net
```

Same network → Docker DNS answers `shop` . Default bridge → no name resolution.

Verification

- ☐ I can create a network and attach containers to it.
- ☐ I can reach a container by its name from another container.
- ☐ I know why names do not work on the default bridge.
- ☐ I can find out which networks a container is on.

Real-World Usage

Service-to-service communication is everyday DevOps work: a web tier calls an API, the API calls a database, a cache or a message queue. Network segmentation (frontend/backend) is a basic security control in every environment, from Docker Compose on a laptop to large container platforms later in your career.

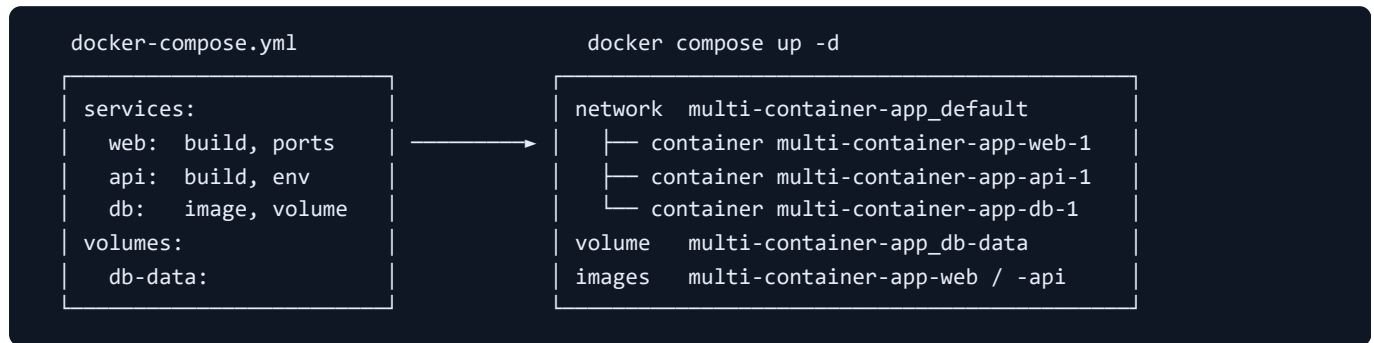
Key Takeaways

- Containers on the same user-defined network reach each other **by name**.
- The default `bridge` network has no name resolution.
- `localhost` inside a container means that container.
- Separate networks = separate trust zones.

Docker Compose

What is it?

Docker Compose lets you describe a whole application (several containers, their networks, volumes, ports and environment variables) in **one YAML file**, and start or stop all of it with one command.



Why do we need it?

In [labs/12-multi-container](#) you start the message board by hand: create a network, create a volume, then three long `docker run` commands in the right order with the right flags. It works, but it is easy to get wrong and impossible to remember. A Compose file is that same knowledge, written down once, versioned in Git, and repeatable for everyone on the team.

How does it work?

The `docker compose` command reads `docker-compose.yml` (or `compose.yaml`) in the current folder. Each entry under `services:` becomes one or more containers. Compose also creates:

- a **project name**: the folder name (or `name:` in the file). Everything it creates is prefixed with it.
- a **default network** `<project>_default`, joined by every service, so services reach each other **by service name**.
- **named volumes** listed under `volumes:` as `<project>_<name>`.
- **images** for services with `build:`.

Command	What it does
<code>docker compose up -d</code>	create everything that is missing and start it in the background
<code>docker compose up -d --build</code>	rebuild images first
<code>docker compose ps</code>	the containers of this project
<code>docker compose logs api</code>	logs of one service (<code>-f</code> to follow)
<code>docker compose exec db psql -U board</code>	run a command in a running service container
<code>docker compose build</code>	only build the images
<code>docker compose restart api</code>	restart one service
<code>docker compose down</code>	remove containers and networks (volumes are kept)

Command	What it does
<code>docker compose down -v</code>	...and the named volumes too (data is deleted)

Prerequisites

- [09-dockerfile.md](#), [12-volumes.md](#), [14-networking.md](#)
- [labs/12-multi-container](#) (doing it by hand first makes Compose obvious)

Hands-on Lab

Full lab: [labs/13-docker-compose](#). Short version:

```
cd examples/multi-container-app
```

```
docker compose up -d --build
```

```
...
Container multi-container-app-web-1 Creating
Container multi-container-app-web-1 Created
Container multi-container-app-db-1 Starting
Container multi-container-app-db-1 Started
Container multi-container-app-api-1 Starting
Container multi-container-app-api-1 Started
Container multi-container-app-web-1 Starting
Container multi-container-app-web-1 Started
```

```
docker compose ps
```

NAME STATUS	IMAGE PORTS	COMMAND	SERVICE	CREATED
multi-container-app-api-1 Up Less than a second	multi-container-app-api 5000/tcp	"gunicorn --bind 0.0..."	api	2 seconds ago
multi-container-app-db-1 Up 1 second	postgres:18-alpine 5432/tcp	"docker-entrypoint.s..."	db	2 seconds ago
multi-container-app-web-1 Up Less than a second	multi-container-app-web 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	"/docker-entrypoint...."	web	2 seconds ago

```
curl -s http://localhost:8080/api/health
```

Expected Result

- `up` builds the `web` and `api` images, creates the network and the volume, and starts `db`, `api`, `web` in that order (because of `depends_on`).
- `ps` shows three containers, all `Up`, and only `web` with a published port (`0.0.0.0:8080->80/tcp`).
- The health endpoint answers `{"database":"ok","status":"ok"}` : browser → web → api → db works.
- Open <http://localhost:8080> in your browser to see the message board.

Experiment

See exactly what Compose created:

```
docker network ls --filter name=multi-container-app
```

```
docker volume ls --filter name=multi-container-app
```

Run a command inside a service container (here: ask PostgreSQL how many messages it stores):

```
docker compose exec -T db psql -U board -d board -c "SELECT count(*) FROM messages"
```

(`-T` turns off the interactive terminal; you only need it in scripts. Typing it by hand works with or without.)

Break It

`down` removes containers and the network but **keeps** the volume. `down -v` deletes the volume as well. Let's prove the first one keeps your data:

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST -H "Content-Type: application/json" -d
'{"text":"survive me"}' http://localhost:8080/api/messages
docker compose down
docker compose up -d
```

```
curl -s http://localhost:8080/api/messages
```

Troubleshoot It

When a Compose application misbehaves, check in this order:

```
docker compose ps -a
```

```
docker compose logs api
```

```
...
api-1 | [2026-10-04 05:18:51 +0000] [1] [INFO] Using worker: sync
api-1 | [2026-10-04 05:18:51 +0000] [7] [INFO] Booting worker with pid: 7
api-1 | [2026-10-04 05:18:51 +0000] [8] [INFO] Booting worker with pid: 8
api-1 | [2026-10-04 05:18:51 +0000] [1] [INFO] Control socket listening at /root/.gunicorn/gunicorn.ctl
api-1 | 172.18.0.4 - - [04/Oct/2026:05:18:51 +0000] "GET /api/messages HTTP/1.1" 200 205 "-"
"curl/8.19.0"
```

`ps -a` shows stopped containers too (look for `Exited (N)`), and `logs <service>` shows why. Real examples: [troubleshooting/06-environment-variable-missing](#) and [troubleshooting/03-containers-cannot-communicate](#).

Clean up. `-v` deletes the database volume, so only use it when you really want the data gone:

```
docker compose down -v
```

Common Mistakes

- Running `docker compose` from the wrong folder ("no configuration file provided").
- Forgetting `--build` after changing code: Compose reuses the old image.
- Using `down -v` out of habit and losing database data.
- Thinking `depends_on` waits until a service is **ready**. By default it only waits until it is **started**. The capstone uses `condition: service_healthy` with health checks for that.
- Indentation errors in YAML (use spaces, never tabs).

Best Practices

- Keep the Compose file in Git next to the code.
- Publish only the ports people need.
- Use named volumes for data, health checks for readiness, and `.env` files for settings that change per machine.
- Keep secrets out of the YAML (the capstone uses `secrets:` with a file that is not committed).

Challenge

Task: change the greeting the API returns to `Hello from my own Compose file` **without editing the YAML**, using an override on the command line.

Hints: `docker compose run` does not help here. Look at how `GREETING` is set and remember that Compose can read another file with `-f`.

Solution

▼ Solution

In `examples/multi-container-app`, create an override file next to the original (Compose merges files given with `-f`, later files win):

```
printf 'services:\n  api:\n    environment:\n      GREETING: Hello from my own Compose file\n' >
greeting.override.yml
docker compose -f docker-compose.yml -f greeting.override.yml up -d --build
```

```
curl -s http://localhost:8080/api/info
```

```
docker compose -f docker-compose.yml -f greeting.override.yml down -v
rm greeting.override.yml
```

Override files are how teams keep one base file and small per-environment differences.

Verification

- [] I can start, inspect and stop a multi-container app with Compose.
- [] I can name everything `docker compose up` created.

- [] I know the difference between `down` and `down -v`.
- [] I can read the logs of one service and run a command inside it.

Real-World Usage

Compose is the standard tool for **local development environments**: a new team member clones the repo, runs `docker compose up`, and has the app plus its database running in minutes. It is also used for demos, integration tests and small single-server deployments.

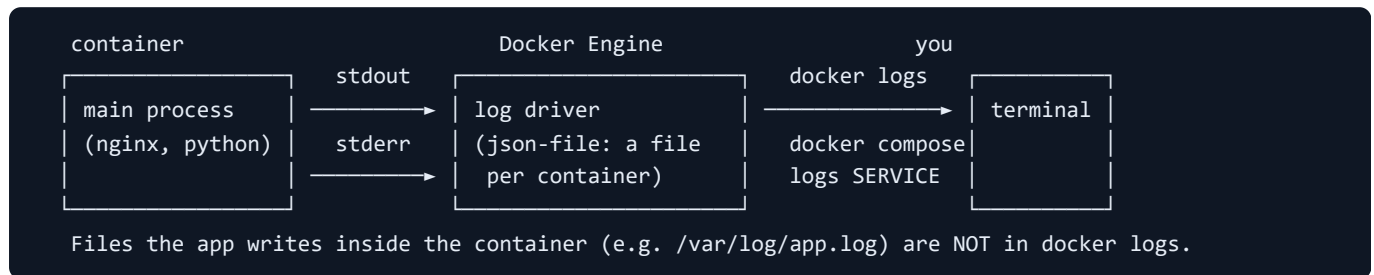
Key Takeaways

- One YAML file describes containers, networks, volumes and configuration.
- Services find each other by **service name** on the project network.
- `down` keeps volumes; `down -v` deletes them.
- `ps -a + logs <service>` is the first step of every Compose investigation.

Logs and Debugging

What is it?

Whatever the main process of a container writes to **standard output (stdout)** or **standard error (stderr)** is captured by Docker. `docker logs` shows it. Logs are the first and most important evidence when something goes wrong, together with `docker ps -a` (did it stop? with which exit code?) and `docker exec` (look inside).



Why do we need it?

Containers are not machines you log into. When a container crashes it may already be gone, but its logs stay until you remove it. A good debugging habit replaces guessing:

```
Observe → Investigate → Identify root cause → Fix → Verify
docker ps -a    docker logs    read the error    change    check again
                docker inspect  message carefully one thing
                docker exec
```

How does it work?

Command	What it shows
<code>docker logs NAME</code>	everything the container printed since it started
<code>docker logs --tail 20 NAME</code>	only the last 20 lines
<code>docker logs --since 5m NAME</code>	only the last 5 minutes
<code>docker logs -t NAME</code>	with timestamps
<code>docker logs -f NAME</code>	follow live (Ctrl+C to stop following; the container keeps running)
<code>docker compose logs SERVICE</code>	the same for a Compose service (<code>-f</code> to follow)
<code>docker ps -a</code>	stopped containers too, with <code>Exited (CODE)</code>
<code>docker exec -it NAME sh</code>	a shell inside a running container

Exit codes worth knowing: `0` finished normally, `1 / 2 / 3` the program reported an error, `126 / 127` the command could not be executed or was not found, `137` killed (by `docker kill` or because it ran out of memory), `143` stopped with SIGTERM (`docker stop`).

Prerequisites

- [05-docker-cli.md](#), [06-container-lifecycle.md](#)

Hands-on Lab

Full lab: [labs/14-logs-inspect-resources](#). Short version:

```
docker run -d --name web -p 8080:80 nginx:1.30-alpine
```

```
curl -s http://localhost:8080
```

```
docker logs --tail 3 web
```

```
2026/10/04 05:18:59 [notice] 1#1: start worker process 43
2026/10/04 05:18:59 [notice] 1#1: start worker process 44
172.17.0.1 - - [04/Oct/2026:05:19:00 +0000] "GET / HTTP/1.1" 200 896 "-" "curl/8.19.0" "-"
```

Watching live is how you debug "what happens when I click this?". It never ends by itself, so it is not part of the automatic tests; press **Ctrl+C** to stop following:

```
docker logs -f web
```

Expected Result

The last log lines include an **access log** line for your `curl`: client IP, time, `"GET / HTTP/1.1" 200`, and the user agent `curl/...`. nginx writes access logs to stdout, which is why Docker can show them.

Experiment

A container that crashes still has logs. Let's create one on purpose:

```
docker run --name crasher alpine:3.24 sh -c "echo 'starting the job'; echo 'ERROR: config file /etc/job.conf not found' >&2; exit 3"
```

```
docker ps -a --filter name=crasher --format '{{.Names}} {{.Status}}'
```

```
docker logs crasher
```

The container is stopped, but `docker logs` still shows both lines, the one on stdout and the one on stderr.

Break It

Run a container whose command does not exist:

```
docker run --name typo alpine:3.24 pyhton --version
```

Troubleshoot It

```
docker ps -a --filter name=typo --format '{{.Names}} {{.Status}}'
```

```
docker inspect typo --format 'exit code: {{.State.ExitCode}} error: {{.State.Error}}'
```

The error is in the **state**, not in the logs: the program never started, so it could not print anything. Exit code 127 means "command not found". Root cause: a typo (`python`) and the alpine image has no Python anyway. Fix: run a command that exists in the image, and verify:

```
docker run --rm alpine:3.24 echo Hello
```

```
docker rm -f web_crasher typo
```

More real investigations: [troubleshooting/01-container-exits-immediately](#), [troubleshooting/06-environment-variable-missing](#).

Common Mistakes

- Looking only at `docker ps` (running containers) and concluding "the container disappeared". Use `docker ps -a`.
- Removing a crashed container before reading its logs.
- Expecting logs from files inside the container. Only stdout/stderr reach `docker logs`.
- Changing three things at once. Change one, verify, then the next.

Best Practices

- Make applications log to stdout/stderr (the official nginx and Python images already do).
- Fail fast with a clear message (the message board API prints exactly which setting is missing).
- Use `--tail` and `--since` on busy containers.
- Read the **first** error, not the last one; later errors are often consequences.

Challenge

Task: start `busybox:1.37` with the command `sh -c 'i=0; while true; do i=$((i+1)); echo tick $i; sleep 1; done'` in the background as `ticker`, wait a few seconds, then show **only the last 2 lines** of its logs, with timestamps.

Solution

▼ Solution

```
docker run -d --name ticker busybox:1.37 sh -c 'i=0; while true; do i=$((i+1)); echo tick $i; sleep 1; done'
```

```
docker logs -t --tail 2 ticker
docker rm -f ticker
```

`-t` adds the timestamp Docker recorded for each line; `--tail 2` limits the output.

Verification

- [] I can read the logs of a running and of a stopped container.
- [] I can follow logs live and stop following.
- [] I know where to look when a program never started (exit code 127, `.State.Error`).
- [] I follow Observe → Investigate → Root cause → Fix → Verify.

Real-World Usage

Logs are the first stop in every incident. In production, the same stdout/stderr streams are shipped by a log driver or agent to a central log system, but the habit is identical: find the failing component, read its first error, form one hypothesis, test it.

Key Takeaways

- `docker logs` shows stdout and stderr of the main process, even after it stopped.
- `docker ps -a` + exit code tells you **that** it failed; logs tell you **why**.
- If the program never started, the reason is in `docker inspect ( .State )`, not in the logs.
- Investigate before you change anything.

Docker Inspect

What is it?

`docker inspect` prints **everything Docker knows** about a container (or image, volume, network) as JSON: its configuration, state, IP address, published ports, environment variables, mounts, networks, limits and more. `--format` picks out exactly the field you need.

```
docker inspect web
[
  {
    "Id": "3c6d...",           full container ID
    "State": {                 running? exit code? OOMKilled? health?
      "Status": "running", "ExitCode": 0, "OOMKilled": false, ...
    },
    "Config": {               what the image + docker run asked for
      "Image": "nginx:1.30-alpine",
      "Env": ["APP_ENV=dev", ...],
      "Cmd": ["nginx", "-g", "daemon off;"], ...
    },
    "HostConfig": {           how Docker runs it on this computer
      "PortBindings": {"80/tcp": [{"HostPort": "8080"}]},
      "Memory": 0, "NanoCpus": 0, "ReadonlyRootfs": false, ...
    },
    "Mounts": [ {"Type": "volume", "Name": "...", "Destination": "..."} ],
    "NetworkSettings": {
      "Ports": {...},
      "Networks": { "bridge": {"IPAddress": "172.17.0.2", ...} }
    }
  }
]
```

Why do we need it?

`docker ps` shows a summary. When you need **facts** ("which port is really published?", "which volume is mounted?", "did the container get the environment variable?", "was it killed for using too much memory?"), `inspect` is the source of truth. It replaces guessing with evidence.

How does it work?

`--format` uses Go templates: `{{.Path.To.Field}}`. A few patterns cover almost everything:

You want	Command
status and exit code	<code>docker inspect NAME --format '{{.State.Status}} {{.State.ExitCode}}'</code>
IP address (default bridge)	<code>docker inspect NAME --format '{{.NetworkSettings.IPAddress}}'</code>
IP on every network	<code>--format '{{range \$n, \$c := .NetworkSettings.Networks}}{{ \$n }}={{ \$c.IPAddress }}{{end}}'</code>
published ports	<code>--format '{{json .NetworkSettings.Ports}}'</code>

You want	Command
environment variables	<code>--format '{{json .Config.Env}}'</code>
mounts	<code>--format '{{json .Mounts}}'</code>
memory limit (bytes)	<code>--format '{{.HostConfig.Memory}}'</code>

`{{json ...}}` prints a whole section as JSON, useful when you are not sure of the field names yet.

Prerequisites

- [05-docker-cli.md](#), [07-ports.md](#), [12-volumes.md](#)

Hands-on Lab

Full lab: [labs/14-logs-inspect-resources](#). Start one container with a port, an environment variable and a volume, then find each one with `inspect`:

```
docker run -d --name web -p 8080:80 -e APP_ENV=dev -v web-logs:/var/log/nginx nginx:1.30-alpine
```

```
docker inspect web --format 'status={{.State.Status}} image={{.Config.Image}}'
```

```
status=running image=nginx:1.30-alpine
```

```
docker inspect web --format '{{.NetworkSettings.Networks.bridge.IPAddress}}'
```

```
172.17.0.2
```

```
docker inspect web --format '{{json .NetworkSettings.Ports}}'
```

```
docker inspect web --format '{{json .Config.Env}}'
```

```
docker inspect web --format '{{range .Mounts}}{{.Type}} {{.Name}} -> {{.Destination}}{{end}}'
```

Expected Result

- Status `running` and the image `nginx:1.30-alpine`.
- An IP address like `172.17.0.x` (yours may differ).
- Ports JSON containing `"HostPort": "8080"` for `80/tcp`.
- The environment list contains `APP_ENV=dev` plus the variables the image defines (`PATH`, `NGINX_VERSION`, ...).
- One mount: `volume web-logs -> /var/log/nginx`.

Experiment

`inspect` works on other objects too:

```
docker volume inspect web-logs --format '{{.Mountpoint}}'
```

```
docker image inspect nginx:1.30-alpine --format '{{json .Config}}'
```

The volume lives in Docker's own storage area; the image config shows the default command (`Cmd`) and exposed ports that every container from this image inherits.

Break It

Ask for a field that does not exist:

```
docker inspect web --format '{{.State.Colour}}'
```

Troubleshoot It

Template errors say **which** field is wrong. When you do not know the name, print the parent section as JSON and look:

```
docker inspect web --format '{{json .State}}'
```

There is no `Colour`; the field you probably wanted is `Status` (or `Health.Status` for containers with a health check). Verify:

```
docker inspect web --format '{{.State.Status}}'
```

```
docker rm -f web
docker volume rm web-logs
```

Common Mistakes

- Field names are case-sensitive: `.state.status` does not work.
- Using `.NetworkSettings.IPAddress` for containers on user-defined networks: it is empty there. Use the `Networks` map instead.
- Reading ports from `HostConfig.PortBindings` (what was **requested**) instead of `NetworkSettings.Ports` (what is **active**).
- Treating `inspect` output as a secret-free zone: environment variables, including passwords, are visible to anyone who can run `docker inspect` (see [19-security-basics.md](#)).

Best Practices

- Start with `{{json .Section}}`, then narrow down to the exact field.
- Use `inspect` in scripts instead of parsing `docker ps` text (the capstone's `verify.sh` does this).
- Inspect the **image** before running it, to know its default user, command and ports.

Challenge

Task: for the running container below, find (1) the mapped host port, (2) the value of `GREETING`, (3) the container's IP address, using only `docker inspect`.

```
docker run -d --name quiz -p 8082:80 -e GREETING=hello-inspect nginx:1.30-alpine
```

Solution

▼ Solution

```
docker inspect quiz --format '{{(index (index .NetworkSettings.Ports "80/tcp") 0).HostPort}}'
```

```
docker inspect quiz --format '{{range .Config.Env}}{{println .}}{{end}}'
```

```
docker inspect quiz --format '{{.NetworkSettings.Networks.bridge.IPAddress}}'
```

```
docker rm -f quiz
```

`index` reads a map entry whose key has special characters (`80/tcp`); `range ... println` prints one variable per line.

Verification

- [] I can find a container's IP, ports, environment variables and mounts.
- [] I can print a whole section as JSON to discover field names.
- [] I know `inspect` also works for images, volumes and networks.

Real-World Usage

Runtime investigation: "Is the config what we think it is?" is the first question in many incidents. Engineers use `inspect` to confirm image versions, environment, limits, health and mounts, and in scripts that check deployments automatically.

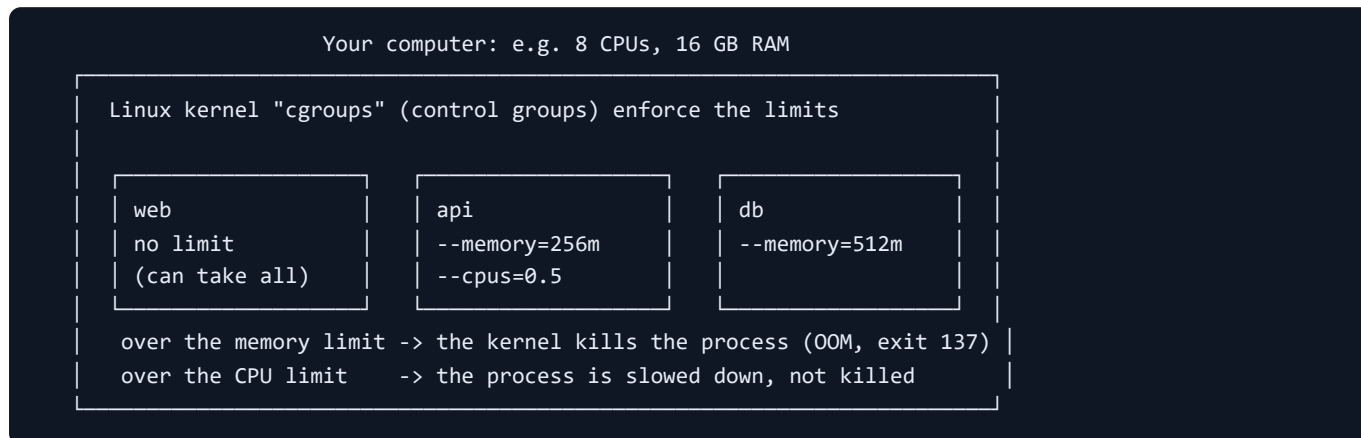
Key Takeaways

- `docker inspect` is the full truth about a Docker object.
- `--format '{{...}}'` extracts single fields; `{{json ...}}` shows sections.
- Requested settings live in `Config / HostConfig`; live state in `State / NetworkSettings`.

Resource Management

What is it?

Containers share your computer's CPU and memory. By default a container may use **as much as it wants**. Docker can **measure** usage (`docker stats` , `docker top`) and **limit** it (`--memory` , `--cpus`), so one misbehaving container cannot slow down or crash everything else.



Why do we need it?

- A memory leak in one container should kill **that** container, not your whole machine.
- Limits make behaviour predictable: what you test locally behaves the same on a shared server.
- Measuring tells you what a service really needs before you set limits.

How does it work?

Command / flag	What it does
<code>docker stats</code>	live CPU %, memory usage / limit, network I/O, block I/O, process count (Ctrl+C to stop)
<code>docker stats --no-stream</code>	one snapshot, then exit (good for scripts)
<code>docker top NAME</code>	the processes running inside the container
<code>--memory=256m</code>	hard memory limit; above it the process is killed (OOMKilled , exit code 137)
<code>--cpus=0.5</code>	at most half of one CPU core; the process is throttled, never killed
Compose <code>deploy.resources.limits</code>	the same limits in a Compose file (the capstone uses them)

Columns of `docker stats` : **CPU %** (100 % = one full core), **MEM USAGE / LIMIT** (limit = your machine's memory if unset), **MEM %**, **NET I/O** (bytes received / sent), **BLOCK I/O** (disk read / written), **PIDS** (processes and threads).

Prerequisites

- [05-docker-cli.md](#), [17-docker-inspect.md](#)

Hands-on Lab

Full lab: [labs/14-logs-inspect-resources](#). Short version:

```
docker run -d --name limited --memory=256m --cpus=0.5 nginx:1.30-alpine
```

```
docker stats --no-stream limited
```

CONTAINER ID	NAME	CPU %	MEM USAGE / LIMIT	MEM %	NET I/O	BLOCK I/O	PIDS
b14c5de1afc4	limited	0.00%	11.64MiB / 256MiB	4.55%	690B / 84B	0B / 8.19kB	15

```
docker top limited
```

```
docker inspect limited --format 'memory={{.HostConfig.Memory}} nanocpus={{.HostConfig.NanoCpus}}'
```

Expected Result

- `stats` shows the `limited` container with a **MEM LIMIT of 256MiB** (instead of your machine's total memory) and a low CPU %. nginx is idle.
- `top` lists the nginx **master** process and its **worker** processes.
- `inspect` shows the limits in bytes and nano-CPU: 268435456 bytes = 256 MiB, 500000000 = 0.5 CPU.

Experiment

Live view (stop with Ctrl+C). Useful while you load the app in a browser:

```
docker stats
```

Break It

Give a Python program 64 MiB and ask it to allocate 256 MiB:

```
docker run --name oom --memory=64m --memory-swap=64m python:3.14-slim python -c "x = bytearray(256*1024*1024)"
```

`--memory-swap=64m` (the same value as `--memory`) means "no swap on top". Without it, on a machine with swap the container may move memory to disk instead of being stopped, and the demo just runs slowly.

Troubleshoot It

The command failed and printed nothing useful. Observe the exit code and the state:

```
docker inspect oom --format 'exit={{.State.ExitCode}} OOMKilled={{.State.OOMKilled}}'
```

Exit code **137** plus **OOMKilled=true** is the fingerprint of "killed for using too much memory". Python had no chance to log anything: the kernel killed it. Fix: give it enough memory (or fix the program), then verify:

```
docker run --rm --memory=512m python:3.14-slim python -c "x = bytearray(256*1024*1024);
print('allocated', len(x) // 1024 // 1024, 'MiB')"
```

```
docker rm -f limited oom
```

Common Mistakes

- Setting limits without measuring first: a database with 64 MiB will be killed over and over.
- Confusing CPU and memory behaviour: too little CPU = slow; too little memory = killed.
- Forgetting that `docker stats` without `--no-stream` never ends (press Ctrl+C).
- Missing the OOM fingerprint and searching the application logs for an error that was never written.

Best Practices

- Watch `docker stats` under realistic load, then set the memory limit with headroom (for example 1.5–2× normal use).
- Always limit memory for services that can grow (caches, workers, databases).
- Use CPU limits to keep noisy background jobs from slowing down interactive services.
- Put limits in the Compose file so everyone runs with the same values.

Challenge

Task: run `busybox:1.37` with the command `sh -c 'while true; do ;; done'` (a busy loop) as `burner` with `-cpus=0.25`, and confirm with one `docker stats --no-stream` snapshot that it stays around 25 % CPU.

Solution

▼ Solution

```
docker run -d --name burner --cpus=0.25 busybox:1.37 sh -c 'while true; do ;; done'
```

```
docker stats --no-stream --format '{{.Name}} {{.CPUPerc}}' burner
docker rm -f burner
```

Without the limit, the loop would use 100 % of a core. With `--cpus=0.25` the kernel lets it run only a quarter of the time. (The exact number varies a little between snapshots.)

Verification

- [] I can read every column of `docker stats`.
- [] I can list the processes of a container.
- [] I can set and verify memory and CPU limits.
- [] I recognise an out-of-memory kill (137 + OOMKilled=true).

Real-World Usage

Capacity planning and stability: on shared servers and container platforms, every service gets requests and limits so that one bad deploy cannot take down its neighbours. "Exit 137 / OOMKilled" is one of the most common production incidents you will investigate.

Key Takeaways

- Measure with `docker stats` and `docker top`, then limit with `--memory` and `--cpus`.
- Memory limit exceeded → killed (137, OOMKilled). CPU limit → throttled.
- Limits belong in the Compose file, next to the service.

Security Basics

What is it?

Beginner-level Docker security is a handful of habits that remove the most common risks:

1. **Do not run as root** inside the container (`USER` in the Dockerfile).
2. **Use small, trusted, pinned images** (official images, a specific tag like `python:3.14-slim`).
3. **Scan images** for known vulnerabilities.
4. **Keep secrets out of environment variables and images** (use files / Docker secrets).
5. **Least privilege**: read-only filesystem, no extra Linux capabilities, publish only needed ports.

This lesson is deliberately basic. It gives you the right instincts, not an advanced security course.

Why do we need it?

A container is isolated, but not a security boundary as strong as a virtual machine: it shares the host's kernel. If an attacker breaks into your application, what they can do next depends on these settings. Root user + writable filesystem + all capabilities + a password in an environment variable is the attacker's best case. Every habit above takes something away from them.

How does it work?

Dockerfile / docker run

```
FROM python:3.14-slim
RUN useradd ... appuser
USER appuser
--read-only
--cap-drop ALL
secrets as files
publish only the web port
```

What it takes away from an attacker

```
fewer packages = fewer vulnerabilities, no compilers

not root: cannot change system files or install tools
cannot modify the app or drop malware on disk
root (if any) loses special kernel powers (chown, raw sockets, ...)
passwords are not visible in docker inspect / child processes
database and API are not reachable from outside
```

Root vs non-root. By default, the process in most images runs as **root (uid 0)**. `USER appuser` in the Dockerfile (or `--user` at run time) changes that. The simple-app and capstone images already do it.

Secrets vs environment variables. Environment variables are easy, but anyone who can run `docker inspect` (and every child process) can read them. Docker Compose `secrets:` mounts the value as a **file** in `/run/secrets/` instead. The official PostgreSQL image reads `POSTGRES_PASSWORD_FILE`; the message board API reads `DB_PASSWORD_FILE`.

Image scanning. A scanner compares the packages inside an image with public vulnerability databases. Docker Desktop includes Docker Scout; the free open-source scanner Trivy runs as a container. Scanning is a concept to know here; it is optional in this course.

Prerequisites

- [09-dockerfile.md](#), [08-environment-variables.md](#)

Hands-on Lab

Full lab: [labs/15-security](#). Who am I in a default container?

```
docker run --rm alpine:3.24 whoami
```

Now the simple-app image, which has `USER appuser` in its Dockerfile:

```
cd examples/simple-app
docker build -t simple-app:1.0 .
```

```
docker run --rm simple-app:1.0 id
```

Expected Result

- `alpine` answers `root`.
- `simple-app` answers `uid=10001(appuser) gid=10001(appuser) ...`: the application runs as an ordinary user.

Experiment

Read-only filesystem: the container cannot write anywhere (except mounted volumes or tmpfs).

```
docker run --rm --read-only alpine:3.24 touch /hacked
```

Dropping capabilities: even root loses special powers such as changing file owners.

```
docker run --rm --cap-drop ALL alpine:3.24 chown nobody /tmp
```

Optional: scan an image with Trivy (downloads a vulnerability database; takes a minute the first time):

```
docker run --rm -v /var/run/docker.sock:/var/run/docker.sock aquasec/trivy:0.75.0 image --severity HIGH,CRITICAL simple-app:1.0
```

Break It

Pass a password the "easy" way:

```
docker run -d --name leaky -e DB_PASSWORD=super-secret-123 nginx:1.30-alpine
```

```
docker inspect leaky --format '{{json .Config.Env}}'
```

Anyone with access to the Docker engine can read it, and it often ends up in logs and screenshots.

Troubleshoot It

The fix is to pass the secret as a **file**, mounted read-only, and let the program read the file. A simple version with a bind mount (Compose `secrets:` does the same thing for you):

```
docker rm -f leaky
printf 'super-secret-123' > db_password.txt
docker run -d --name safer -v "$(pwd)/db_password.txt":/run/secrets/db_password:ro -e
DB_PASSWORD_FILE=/run/secrets/db_password nginx:1.30-alpine
```

```
docker inspect safer --format '{{json .Config.Env}}'
```

```
docker exec safer cat /run/secrets/db_password
```

Verified: `inspect` only shows the **path** of the secret, while the file is still available to the process that needs it. Clean up (and never commit secret files: list them in `.gitignore`):

```
docker rm -f safer
rm db_password.txt
```

Common Mistakes

- `FROM something:latest` from an unknown publisher.
- Copying `.env` or private keys into the image with `COPY . .` (use `.dockerignore`, see [11-dockerignore.md](#)).
- Putting passwords in `ENV` inside the Dockerfile: they are stored in the image forever (`docker history` shows them).
- Running as root "because it is easier", or adding `--privileged` to make an error go away.
- Publishing the database port to the whole network.

Best Practices

- Official or verified images, pinned to a version tag.
- `USER` with a fixed numeric uid in every Dockerfile you write.
- `--read-only` (+ `tmpfs` for `/tmp`) and `--cap-drop ALL` where the app allows it; the capstone API runs read-only.
- Secrets as files, never in images, never in Git.
- Scan images regularly and rebuild them to pick up security updates.

Challenge

Task: run `nginx:1.30-alpine` so that its filesystem is read-only, yet it still starts and serves the welcome page on port 8081.

Hints: nginx writes temporary files to `/var/cache/nginx` and its PID file to `/run`. `--tmpfs PATH` gives a container a small writable in-memory folder.

Solution

▼ Solution

```
docker run -d --name ro-web -p 8081:80 --read-only --tmpfs /var/cache/nginx --tmpfs /run nginx:1.30-alpine
```

```
curl -s http://localhost:8081
```

```
docker exec ro-web touch /usr/share/nginx/html/hacked
```

```
docker rm -f ro-web
```

Without the two `--tmpfs` folders nginx exits at start-up because it cannot create its cache and PID files. Try it: `docker logs` will tell you exactly which path it needed.

Verification

- [] I can check which user a container runs as, and change it with `USER` / `--user`.
- [] I can run a container with a read-only filesystem and without capabilities.
- [] I know why passwords do not belong in environment variables or images.
- [] I know what an image scanner does.

Real-World Usage

Security reviews and platform policies check exactly these points: non-root, pinned base images, no secrets in images, scanned images, minimal privileges. Getting them right from your first Dockerfile saves painful rework later.

Key Takeaways

- Containers run as root by default; change it with `USER`.
- Small, official, pinned images; scan them.
- Secrets as files, not environment variables, never in images.
- Read-only filesystem and dropped capabilities limit the damage of a break-in.

Image Optimization

What is it?

Making images **smaller and faster to build** without changing what the application does. The main tools:

- a **smaller base image** (`-slim` or `-alpine` instead of the full image)
- **multi-stage builds** (build tools in one stage, only the result in the final image)
- `.dockerignore` (do not send junk into the build)
- **layer order** (dependencies first, code last, so the cache works)
- **no unnecessary packages or files** (no `vim` "just in case", no package-manager caches)

Why do we need it?

Every megabyte is pulled by every server, every CI job and every teammate, every time the image changes. Smaller images download faster, start faster, cost less storage, and contain **fewer packages that can have vulnerabilities**. Faster builds mean faster feedback for developers.

How does it work?

Base images. The same Python, three sizes:

python:3.14	Debian + compilers + headers + docs	very large
python:3.14-slim	Debian, only what Python needs	small
python:3.14-alpine	Alpine Linux (musl), minimal	smallest

Multi-stage build. One Dockerfile, several `FROM` lines. Only the **last** stage becomes the image; earlier stages are thrown away after you copy what you need with `COPY --from=`:

```
Stage 1: builder (FROM python:3.14-alpine AS builder)

| build-base (gcc, make, headers ...) | discarded
| python -m venv /opt/venv              |
| pip install -r requirements.txt       |
|                                     |
|                                     | COPY --from=builder /opt/venv /opt/venv
|                                     |
Stage 2: runtime (FROM python:3.14-alpine) ▼
| /opt/venv (Flask installed)           | this is your image
| app.py, USER appuser                 |
```

The repository contains three versions of the simple-app image to compare:

File	Base	What it does wrong / right
examples/simple-app/optimization/fat.Dockerfile	python:3.14	full image, <code>apt-get install build-essential curl vim</code> , <code>apt lists</code> and <code>pip cache kept</code> , <code>COPY . .</code> before <code>pip install</code>

File	Base	What it does wrong / right
examples/simple-app/Dockerfile	python:3.14-slim	slim base, dependencies first, <code>--no-cache-dir</code> , non-root
examples/simple-app/optimization/multistage.Dockerfile	python:3.14-alpine ×2	builder stage with compilers, runtime stage with only the venv and <code>app.py</code>

Prerequisites

- [09-dockerfile.md](#), [10-image-layers.md](#), [11-dockerignore.md](#)

Hands-on Lab

Full lab: [labs/16-image-optimization](#). Build all three (the fat one takes a few minutes the first time; that slowness is part of the lesson):

```
cd examples/simple-app
```

```
docker build -f optimization/fat.Dockerfile -t simple-app:fat .
```

```
docker build -t simple-app:slim .
docker build -f optimization/multistage.Dockerfile -t simple-app:multistage .
```

```
docker images simple-app --format 'table {{.Repository}}:{{.Tag}}\t{{.Size}}'
```

```
REPOSITORY:TAG      SIZE
simple-app:step5      212MB
simple-app:step3      212MB
simple-app:multistage 108MB
simple-app:fat         1.75GB
simple-app:mime        212MB
simple-app:slim        212MB
simple-app:1.0         212MB
simple-app:leaky       239MB
simple-app:step4       212MB
simple-app:fixed       212MB
simple-app:step2       189MB
simple-app:clean       189MB
simple-app:step1       189MB
```

Expected Result

The `fat` image is **many times** the size of the others. `slim` is a fraction of it, and `multistage` (Alpine based, no compilers) is the smallest. All three run the same application. (You may also see other `simple-app` tags from earlier labs; compare the three tags you just built.)

Experiment

Prove they behave the same:

```
docker run -d --name small -p 5000:5000 simple-app:multistage
```

```
curl -s http://localhost:5000
```

And see **where** the size of the fat image comes from, layer by layer:

```
docker history simple-app:fat --format 'table {{.Size}}\t{{.CreatedBy}}'
```

SIZE	CREATED BY
0B	CMD ["python" "app.py"]
18.4MB	RUN /bin/sh -c pip install -r requirements.t...
16.4kB	COPY . . # buildkit
8.19kB	WORKDIR /app
74.5MB	RUN /bin/sh -c apt-get update && apt-get ins...
0B	CMD ["python3"]
16.4kB	RUN /bin/sh -c set -eux; for src in idle3 p...
...	

Read the sizes from the top (newest layer) down. Most of the weight comes from the layers of the full `python:3.14` base image itself, and the `apt-get install` layer adds more on top. Your own code is a few kilobytes.

Break It

The multi-stage file copies the venv from the builder. Ask Docker to copy from a stage that does not exist:

```
sed 's/--from=builder/--from=buildr/' optimization/multistage.Dockerfile > broken.Dockerfile
```

```
docker build -f broken.Dockerfile -t simple-app:broken .
```

Troubleshoot It

Read the build error: Docker tries to treat `buildr` as an **image name** to pull, because no stage is called that, and fails. Compare the stage names:

```
grep -n "AS \|--from" broken.Dockerfile
```

Root cause: the name after `--from=` must match a name after `AS`. Fix by using the original file, and verify:

```
rm broken.Dockerfile
docker build -f optimization/multistage.Dockerfile -t simple-app:multistage .
```

```
docker rm -f small
```

Common Mistakes

- `apt-get update` and `apt-get install` in different `RUN` lines (stale cache) and never cleaning `/var/lib/apt/lists`.

- Installing debugging tools "just in case" into production images.
- `COPY . .` as the first instruction: every code change reinstalls all dependencies.
- Deleting files in a later `RUN`: the earlier layer still contains them, so the image does not shrink.
- Switching to Alpine without testing: some Python packages need extra work on musl. Test, do not assume.

Best Practices

- Start from `-slim`; try Alpine or multi-stage when size matters and the app supports it.
- Dependencies first, code last.
- `--no-cache-dir` for pip, `--no-install-recommends` and `rm -rf /var/lib/apt/lists/*` for apt.
- Multi-stage whenever you need compilers or build tools.
- Keep `.dockerignore` up to date.
- Measure: `docker images` and `docker history` before and after every change.

Challenge

Task: the fat image keeps the pip download cache. Without changing anything else, how much smaller does it get if `pip install` uses `--no-cache-dir`? Build it as `simple-app:fat-nocache` and compare.

Solution

▼ Solution

```
sed 's/pip install -r/pip install --no-cache-dir -r/' optimization/fat.Dockerfile > fat-nocache.Dockerfile
```

```
docker build -f fat-nocache.Dockerfile -t simple-app:fat-nocache .
```

```
docker images simple-app --format '{{.Tag}} {{.Size}}'
rm fat-nocache.Dockerfile
```

The pip cache is a few megabytes here: a small win compared to choosing the right base image. **Optimize the biggest layer first.**

Verification

- [] I can compare image sizes with `docker images` and find the big layer with `docker history`.
- [] I can explain what a multi-stage build keeps and what it throws away.
- [] I can name five ways to make an image smaller.

Real-World Usage

Teams optimize images to speed up CI pipelines and deployments, reduce registry storage and egress costs, and shrink the attack surface that security scanners report on. "Why is this image 1.5 GB?" is a common code review question.

Key Takeaways

- The base image is the biggest lever.
- Multi-stage builds leave build tools behind.
- Layer order decides how much the cache can reuse.
- Measure before and after; optimize the biggest layer first.

Docker Hub and Registries

What is it?

A **registry** is a server that stores images so other computers can download them. **Docker Hub** (<https://hub.docker.com>) is the default public registry: when you run `docker pull nginx:1.30-alpine`, Docker downloads it from Docker Hub.

Vocabulary:

Word	Meaning	Example
registry	the server	<code>docker.io</code> (Docker Hub), <code>localhost:5000</code>
repository	one image name, with all its versions	<code>sufibaba6629/docker-from-zero-web</code>
namespace	the account or organisation that owns repositories	<code>sufibaba6629</code> , <code>library</code> (official images)
tag	a human label for one version	<code>1.0.0</code> , <code>1.30-alpine</code>
digest	the content hash of an image, never changes	<code>sha256:0985e772...</code>
public / private	anyone can pull / only allowed accounts can pull	

A full image name is `REGISTRY/NAMESPACE/REPOSITORY:TAG`. Docker fills in the defaults: `nginx:1.30-alpine` really means `docker.io/library/nginx:1.30-alpine`.

Why do we need it?

An image on your laptop is useless to a server or a teammate. Pushing it to a registry turns it into a **shareable, versioned artifact**: build once, then pull and run the exact same image everywhere.

How does it work?



- `docker tag SOURCE TARGET` only adds a **second name** to the same image (same image ID). The target name decides where `docker push` sends it.
- Push and pull transfer **layers**; layers the other side already has are skipped.
- A tag can be moved to a new image later; a **digest** cannot. Pin by digest when you need to be 100 % sure.
- Docker Hub limits how many images **anonymous** users can pull in a period of time; logged-in (free) accounts get a higher limit. If you see a "rate limit" error, log in or wait.

- `docker login` should use an **access token** (Docker Hub → Account settings → Personal access tokens), not your account password. A token can be limited (read-only, read/write) and revoked at any time.

Prerequisites

- [09-dockerfile.md](#), [03-images.md](#)

Hands-on Lab

Full lab: [labs/17-docker-hub](#). You can practise the whole workflow **without an account** by running your own registry locally (it is just another container):

```
docker run -d --name registry -p 5000:5000 registry:3
docker pull alpine:3.24
docker tag alpine:3.24 localhost:5000/my-alpine:1.0
```

```
docker push localhost:5000/my-alpine:1.0
```

```
curl -s http://localhost:5000/v2/_catalog
```

Now delete the local copy and pull it back from **your** registry:

```
docker rmi localhost:5000/my-alpine:1.0
```

```
docker pull localhost:5000/my-alpine:1.0
```

```
docker run --rm localhost:5000/my-alpine:1.0 echo "pulled from my own registry"
```

macOS: if port 5000 is taken (AirPlay Receiver uses it), use `-p 5001:5000` and `localhost:5001/...` instead.

Expected Result

- `push` prints the layers it uploads and a line with the tag and a `digest: sha256:...`
- The catalog answers `{"repositories":["my-alpine"]}`.
- `pull` downloads the image from `localhost:5000` and `run` works with it.

Experiment

Tags are just names. Both names below point to the **same image ID**:

```
docker images --format '{{.Repository}}:{{.Tag}} {{.ID}}' | grep alpine
```

Break It

Push a name whose registry part points to nothing:

```
docker tag alpine:3.24 localhost:5999/my-alpine:1.0
```

```
docker push localhost:5999/my-alpine:1.0
```

The push retries for a while and then gives up with `dial tcp ...: connect: connection refused` on Linux, or `dial tcp ...: i/o timeout` on Docker Desktop. Both mean the same thing: nothing answered at that address.

Troubleshoot It

The name **is** the destination. `localhost:5999` means "a registry on this computer, port 5999", and nothing is listening there. Check which registries actually run:

```
docker ps --filter name=registry --format '{{.Names}} {{.Ports}}'
```

Fix: tag for the registry that exists (or start one on 5999), then verify:

```
docker rmi localhost:5999/my-alpine:1.0
docker tag alpine:3.24 localhost:5000/my-alpine:1.1
```

```
docker push localhost:5000/my-alpine:1.1
```

On Docker Hub, the same mistake looks like `denied: requested access to the resource is denied`: you tried to push to a namespace that is not yours (or you are not logged in).

```
docker rm -f registry
docker rmi localhost:5000/my-alpine:1.0 localhost:5000/my-alpine:1.1
```

Docker Hub for real

These steps need your own free Docker Hub account, so the automatic tests skip them. Replace `<your-username>` with your Docker Hub **username** (not your e-mail address).

```
docker login -u <your-username>
cd examples/simple-app
docker build -t simple-app:1.0 .
docker tag simple-app:1.0 <your-username>/simple-app:1.0
docker push <your-username>/simple-app:1.0
```

`docker login` asks for a password: paste an **access token**. Then, on any computer:

```
docker pull <your-username>/simple-app:1.0
docker run -d -p 5000:5000 <your-username>/simple-app:1.0
docker logout
```

The author's images for this course are public, so you can pull them without an account:

```
docker pull sufibaba6629/docker-from-zero-web:1.0.0
docker pull sufibaba6629/docker-from-zero-api:1.0.0
```

Common Mistakes

- Pushing without tagging first (`docker push simple-app:1.0` tries `docker.io/library/simple-app`, which is not yours).
- Using the e-mail address instead of the username in the image name.
- Logging in with the account password instead of an access token, and storing it on shared machines.
- Pushing only `latest`: nobody knows which version is running. Use version tags.
- Accidentally pushing an image that contains secrets (`.dockerignore` and [19-security-basics.md](#)).

Best Practices

- Tag with a version (`1.0.0`) and, if you like, also a moving tag (`1.0`).
- Log in with access tokens; `docker logout` on machines you do not own.
- Public repositories for learning material only; private for company images.
- Pull official images by version; for critical deployments, pin by digest.

Challenge

Task: using the local registry, publish `nginx:1.30-alpine` as `localhost:5000/team/web:2.0` and prove (with `docker images`) that it has the same image ID as `nginx:1.30-alpine`.

Solution

▼ Solution

```
docker run -d --name registry -p 5000:5000 registry:3
docker pull nginx:1.30-alpine
docker tag nginx:1.30-alpine localhost:5000/team/web:2.0
```

```
docker push localhost:5000/team/web:2.0
```

```
docker images --format '{{.Repository}}:{{.Tag}} {{.ID}}' | grep -E 'nginx|team/web'
```

```
docker rm -f registry
docker rmi localhost:5000/team/web:2.0
```

Namespaces (`team/`) work the same in your own registry as on Docker Hub.

Verification

- [] I can explain registry, repository, namespace, tag and digest.
- [] I can tag, push, pull and run an image from a registry.

- [] I know to log in with an access token, not my password.

Real-World Usage

Every deployment pipeline ends with "push the image to a registry" and every server starts with "pull the image". Companies use Docker Hub, GitHub Container Registry, or a registry from their cloud provider; the commands are the same.

Key Takeaways

- The image name decides where it is pushed: `REGISTRY/NAMESPACE/REPO:TAG`.
- `docker tag` adds a name; `push` uploads, `pull` downloads only missing layers.
- Tags are labels, digests are fingerprints.
- Use access tokens and version tags.

Troubleshooting

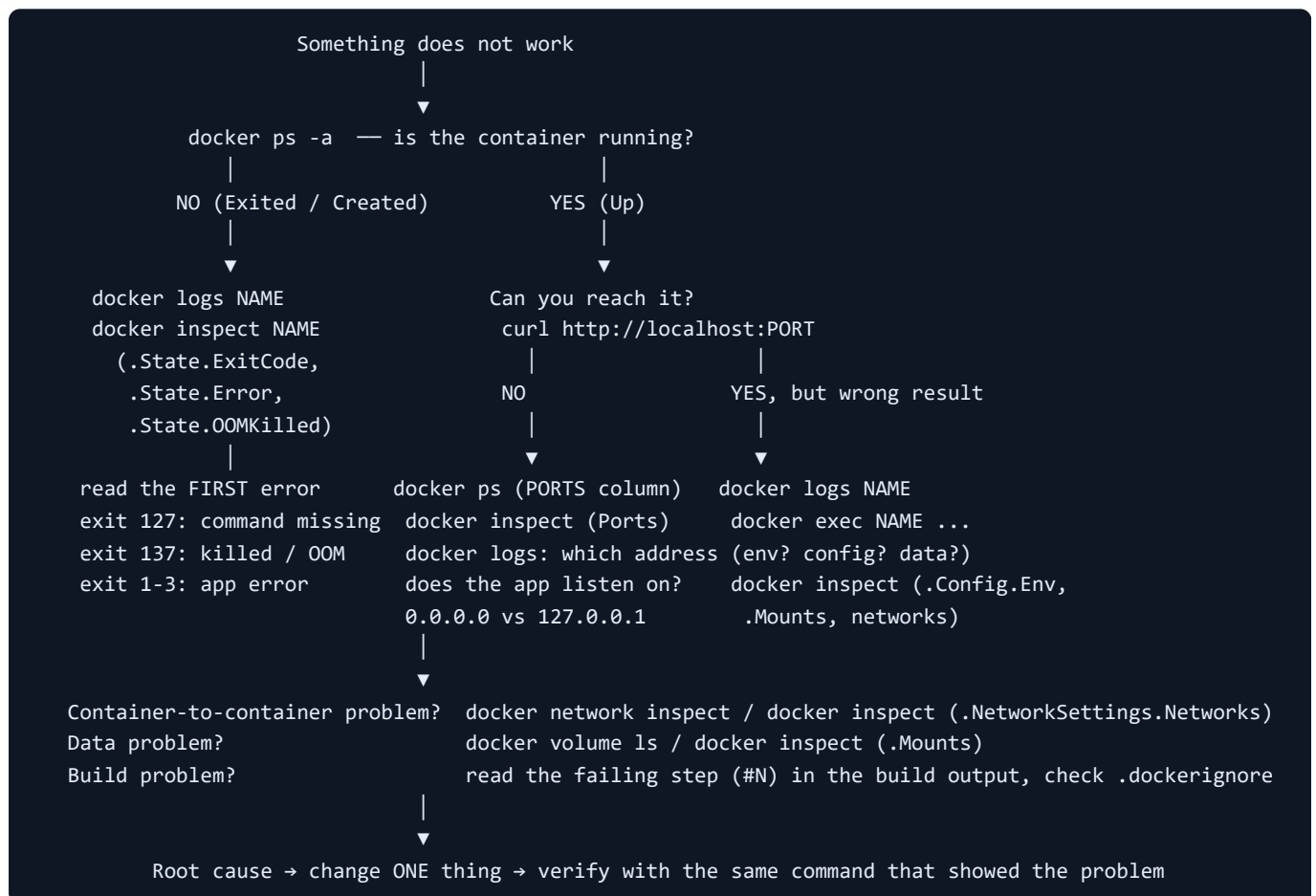
What is it?

Troubleshooting is a **method**, not a list of fixes to memorise. You collect evidence until the cause is obvious, change one thing, and prove the fix worked. This lesson gives you the method and a map of the ten hands-on scenarios in [troubleshooting/](#).

Why do we need it?

Things break all the time: a typo in a Dockerfile, a port already taken, a missing environment variable, a wrong volume name. Beginners guess and change random settings until something works, often making it worse. Engineers ask the system what is wrong. Docker gives you excellent evidence: states, exit codes, logs, inspect output, networks and volumes.

How does it work?



Every scenario in this repository is written with the same eight parts:

Problem → Symptoms → Investigation → Commands → Root Cause → Fix → Verification → Lesson Learned

Prerequisites

- [16-logs-and-debugging.md](#), [17-docker-inspect.md](#)

Hands-on Lab

The ten scenarios. Each folder contains the broken files and a README that walks you through it:

#	Scenario	First command that shows the problem
01	Container exits immediately	<code>docker ps -a</code> , <code>docker logs</code>
02	Port already in use	the <code>docker run</code> error, <code>docker ps --filter publish=8080</code>
03	Containers cannot communicate	<code>docker compose ps -a</code> , <code>docker compose logs web</code>
04	Volume data appears missing	<code>docker volume ls</code> , <code>docker inspect --format '{{json .Mounts}}'</code>
05	Dockerfile build fails	the failing build step, <code>.dockerignore</code>
06	Environment variable missing	<code>docker compose ps -a</code> , <code>docker compose logs api</code>
07	Running but not reachable	<code>curl</code> , <code>docker logs</code> , <code>docker exec</code>
08	Image too large	<code>docker images</code> , <code>docker history</code>
09	Host changes not reflected	<code>docker inspect --format '{{json .Mounts}}'</code>
10	Database data disappears	<code>docker volume ls</code> , <code>docker compose logs db</code>

Let's practise the method once here, on a small example. A job container that should print a report:

```
docker run --name report -e REPORT_NAME= alpine:3.24 sh -c 'test -n "$REPORT_NAME" || { echo "ERROR: REPORT_NAME is empty" >&2; exit 2; }; echo "report $REPORT_NAME done"'
```

Expected Result

The command returns an error and prints `ERROR: REPORT_NAME is empty` . Do not fix it yet. Let's investigate as if you had not seen the message (in real life the container usually runs in the background).

Experiment

Observe:

```
docker ps -a --filter name=report --format '{{.Names}}' '{{.Status}}'
```

Investigate: the logs and the configuration the container really received:

```
docker logs report
```

```
docker inspect report --format '{{range .Config.Env}}{{println .}}{{end}}'
```

Root cause: the variable exists but is empty (`REPORT_NAME=`), and the program checks for that. **Fix** (one change) and **verify** with the same evidence:

```
docker rm report
docker run --name report -e REPORT_NAME=weekly alpine:3.24 sh -c 'test -n "$REPORT_NAME" || { echo "ERROR: REPORT_NAME is empty" >&2; exit 2; }; echo "report $REPORT_NAME done"'
```

```
docker ps -a --filter name=report --format '{{.Names}} {{.Status}}'
```

```
docker logs report
docker rm report
```

Break It

Every scenario in [troubleshooting/](#) is a "break it" exercise with a real broken file. Start with [01](#); it takes ten minutes.

Troubleshoot It

A checklist to keep next to your terminal:

1. `docker ps -a` : running? exit code?
2. `docker logs NAME` (or `docker compose logs SERVICE`): first error message?
3. `docker inspect NAME` : state, env, mounts, ports, networks as Docker sees them.
4. `docker exec NAME ...` : test from **inside** the container (does the app answer on its own port?).
5. `docker network inspect` / `docker volume ls` : are the containers where you think they are?
6. Form one hypothesis, change one thing, re-run the command that showed the problem.

Common Mistakes

- Restarting or recreating before reading the logs (evidence lost).
- Fixing the second error instead of the first.
- Changing several things at once, so you never learn what the cause was.
- Using `--privileged`, `root`, or `docker system prune -a --volumes` to "make it work".
- Not verifying: "it should work now" is not a test.

Best Practices

- Write down what you checked and what you saw; it becomes the incident note or the pull request description.
- Reproduce the problem before fixing it, and keep the reproduction (that is what this repository's scenarios are).
- Make applications fail fast with clear messages (the message board API does).

Challenge

Task: pick any **three** scenarios from the table, and solve them **without opening the Root Cause section** first. Write down the command that gave you the decisive evidence for each.

Solution

▼ Solution

There is no single answer: each scenario's README has its own Investigation, Root Cause and Verification sections to compare your notes with. A good answer names one command per scenario, for example: 01 `docker logs`, 03 `docker compose logs web` + `docker inspect networks`, 07 `docker exec` (it works inside, not outside).

Verification

- [] I follow Observe → Investigate → Root cause → Fix → Verify without skipping steps.
- [] I know which command answers which question (state, logs, config, network, data).
- [] I have solved at least three scenarios from `troubleshooting/`.

Real-World Usage

On-call engineers follow exactly this loop during incidents, and the habit of verifying with the same evidence that showed the problem is what separates a fix from a guess. Teams turn solved problems into runbooks, like the scenario READMEs in this repository.

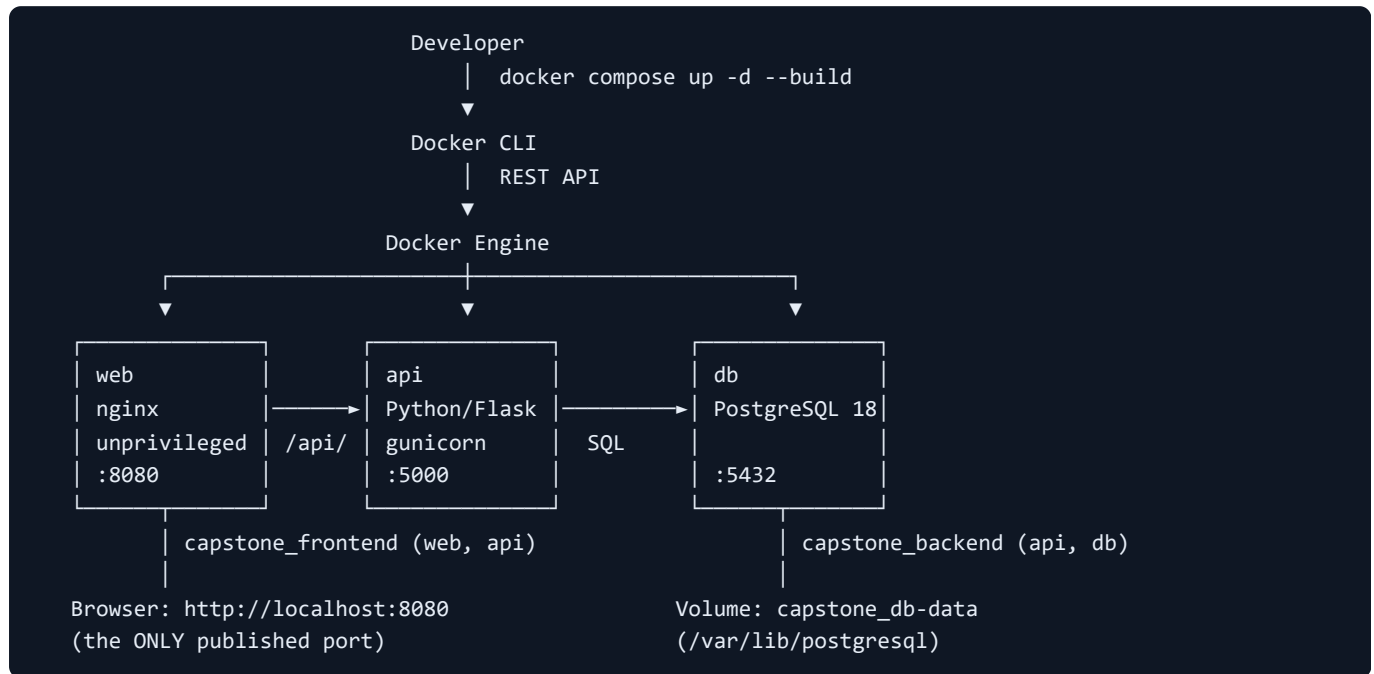
Key Takeaways

- Evidence first: state, logs, inspect, exec, networks, volumes.
- Read the first error; know your exit codes (127, 137, 1–3).
- One change at a time, and always verify.

Capstone: Everything Together

What is it?

The capstone is the message board application built **the way you would build it at work**, using every lesson of this course in one place. It lives in [capstone/](#); the guided walkthrough is [tutorial/10-capstone.md](#).



Why do we need it?

Each lab teaches one idea in isolation. Real work means combining them and making them work **together**: the image must be small and non-root, the containers must find each other on the right networks, the data must survive, the password must not leak, the services must start in the right order, and you must be able to prove all of it.

How does it work?

File	What it does	Lessons it uses
<code>capstone/app/api/Dockerfile</code>	multi-stage build (venv in a builder stage), slim runtime, <code>USER app</code> (uid 10001), <code>HEALTHCHECK</code>	09, 10, 19, 20
<code>capstone/app/api/app.py</code>	the API; reads <code>DB_PASSWORD_FILE</code> , fails fast with a clear message if a setting is missing	08, 16
<code>capstone/app/web/Dockerfile</code>	<code>nginxinc/nginx-unprivileged</code> , listens on 8080, health check	07, 19
<code>capstone/app/web/default.conf.template</code>	forwards <code>/api/</code> to <code>http://\${API_HOST}:5000</code> (a container name, resolved by Docker DNS)	08, 14

File	What it does	Lessons it uses
capstone/database/init.sql	creates the table and the first message, bind-mounted read-only into PostgreSQL	13
capstone/docker-compose.yml	3 services, 2 networks, 1 volume, 1 secret, health-based start order, limits, read-only API	12, 14, 15, 18, 19
capstone/app/*.dockerignore	keep junk and secrets out of the build context	11
capstone/secrets/db_password.txt.example	template for the secret file (the real file is never committed)	19
capstone/.env.example	settings you can change without editing the YAML (WEB_PORT , APP_ENV , GREETING)	08, 15
capstone/verify.sh	18 automatic checks: health, ports, app, networks, security, volume; --persistence adds down + up	16, 17

What Compose creates (project name `capstone`):

```
containers  capstone-web-1  capstone-api-1  capstone-db-1
networks    capstone_frontend  capstone_backend
volume      capstone_db-data
images      docker-from-zero/web:1.0.0  docker-from-zero/api:1.0.0
```

Prerequisites

All labs, or at least: [15-docker-compose.md](#), [14-networking.md](#), [12-volumes.md](#), [19-security-basics.md](#).

Hands-on Lab

```
cd capstone
cp secrets/db_password.txt.example secrets/db_password.txt
```

```
docker compose up -d --build
```

```
...
Container capstone-db-1 Waiting
Container capstone-db-1 Healthy
Container capstone-api-1 Starting
Container capstone-api-1 Started
Container capstone-api-1 Waiting
Container capstone-api-1 Healthy
Container capstone-web-1 Starting
Container capstone-web-1 Started
```

```
./verify.sh
```

Open <http://localhost:8080>, write a message, and keep it in mind for the next step.

Expected Result

- `up` builds two images and starts `db`, then `api` (once `db` is healthy), then `web` (once `api` is healthy).
- `./verify.sh` prints 17 `PASS` lines and `All checks passed.` (18 with `--persistence`).
- The page shows the greeting, the API container's hostname and the messages from the database.

Experiment

Prove that the data lives on the volume, not in the container:

```
./verify.sh --persistence
```

`--persistence` writes a message, runs `docker compose down` (containers and networks are deleted, the volume stays), starts everything again and finds the message.

Break It

Stop the database while the rest keeps running:

```
docker compose stop db
```

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8080/api/health
```

Troubleshoot It

```
docker compose ps -a
```

`db` is `Exited`, the API answers `503` on `/api/health`, and the web page shows an error for the messages. The health endpoint tells you the database is the problem. Fix and verify:

```
docker compose start db
```

```
curl -s http://localhost:8080/api/health
```

Clean up when you are done. `down` keeps your messages; add `-v` only if you want to delete them:

```
docker compose down -v
```

Common Mistakes

- Forgetting to create `secrets/db_password.txt` (Compose stops with an error that names the missing file).
- Port 8080 already used by a container from an earlier lab (`docker ps`, or set `WEB_PORT=8081` in `.env`).
- Mounting PostgreSQL 18 data at `/var/lib/postgresql/data` (old tutorials). It must be `/var/lib/postgresql`.
- Running `docker compose down -v` and wondering where the messages went.

- Expecting to reach the API on `localhost:5000` : it is deliberately not published.

Best Practices

The capstone shows them all in one file: pinned images, non-root users, multi-stage builds, health checks with `depends_on: condition: service_healthy` , separate networks, a single published port, a named volume, secrets as files, a read-only API filesystem, memory and CPU limits, and a script that verifies everything.

Challenge

Task: change the greeting and the published port **without editing** `docker-compose.yml` : the page should be on port 8081 and say `Hello from my capstone` .

Hints: look at `.env.example` .

Solution

▼ Solution

From the `capstone` folder:

```
printf 'WEB_PORT=8081\nGREETING=Hello from my capstone\n' > .env
docker compose up -d
```

```
curl -s http://localhost:8081/api/info
```

```
docker compose down -v
rm .env
```

Compose reads `.env` automatically and substitutes `${WEB_PORT:-8080}` and `${GREETING:-...}` . Only the containers whose configuration changed are recreated.

Verification

- ☐ I can explain what every file in `capstone/` does and which lesson it comes from.
- ☐ I can start, verify, break, fix and stop the whole application.
- ☐ I can prove the data survives `docker compose down` .
- ☐ `./verify.sh` passes on my machine.

Real-World Usage

This is a realistic local environment for a three-tier application, the kind of setup a team keeps in its repository so every developer runs the same stack. The same images, health checks, limits and secret files carry over to the platforms you will meet later in your DevOps career.

Key Takeaways

- Every Docker concept in this course appears in the capstone.

- Health checks + `service_healthy` give a reliable start order.
- Two networks and one published port keep the database private.
- Verify with a script, not with a feeling.

Back to the start: [README](#) · Knowledge check: [tutorial/11-knowledge-check.md](#)

Glossary

Every Docker word used in this lab, in plain language, with where you practise it.

Term	Meaning	Practise it in
Anonymous volume	A volume with a random name, created when an image declares <code>VOLUME</code> and you do not mount anything there. Easy to lose track of.	troubleshooting/10
Base image	The image a Dockerfile starts <code>FROM</code> , e.g. <code>python:3.14-slim</code> .	labs/06
Bind mount	A folder or file from your computer shown inside a container. Changes are visible on both sides immediately.	labs/10
Bridge network	The default kind of Docker network: a private network on one machine that containers join.	labs/11
Build cache	Results of earlier build steps that Docker reuses when nothing they depend on has changed. Shown as <code>CACHED</code> .	labs/07
Build context	The folder you pass to <code>docker build</code> (often <code>.</code>). Everything in it is sent to the builder, minus <code>.dockerignore</code> .	labs/08
BuildKit	The engine that runs <code>docker build</code> in current Docker versions. Its output lines start with <code>#1</code> , <code>#2</code> , ...	labs/06
Capability	A piece of root's power in Linux (for example "change file owners"). <code>--cap-drop ALL</code> removes them all from a container.	labs/15
cgroups	The Linux feature Docker uses to limit memory and CPU (<code>--memory</code> , <code>--cpus</code>).	labs/14
CMD	Dockerfile instruction: the default command of the container. Replaced by anything you type after the image name.	labs/06
Compose	<code>docker compose</code> : describe several containers, networks and volumes in one YAML file and manage them together.	labs/13
Compose project	The group of resources Compose manages together. Its name prefixes containers, networks and volumes.	labs/13
Container	A running (or stopped) instance of an image: an isolated process with its own filesystem, network and settings.	labs/01
Container ID	The unique 64-character hex ID of a container; Docker usually shows the first 12. Also the default hostname.	labs/02
Daemon (dockerd)	The background service that does the real work. The <code>docker</code> command only sends it requests.	docs/01
Digest	<code>sha256:... </code> : the content hash of an image. Unlike a tag, it can never point to different content.	labs/17
Docker Desktop	Docker for Windows and macOS: a small Linux virtual machine plus the Docker tools and a GUI.	docs/02
Docker Engine	Docker on Linux: the daemon, the CLI and the container runtime.	docs/02

Term	Meaning	Practise it in
Docker Hub	The default public registry. <code>nginx</code> really means <code>docker.io/library/nginx</code> .	labs/17
Dockerfile	A text file of instructions that builds an image, one layer per file-changing instruction.	labs/06
.dockerignore	A list of files and folders that are not sent to the build (keeps builds fast and secrets out).	labs/08
ENTRYPOINT	Dockerfile instruction: the fixed program of the container. <code>CMD</code> then supplies its default arguments.	labs/06
Environment variable	A <code>NAME=value</code> setting given to the process, e.g. <code>-e APP_ENV=production</code> .	labs/05
exec	<code>docker exec</code> : run an extra command inside a running container.	labs/02
Exit code	The number a process returns when it ends. 0 = success, 137 = killed (often out of memory), 1/2/3 = errors.	labs/03
EXPOSE	Dockerfile documentation of the port the app listens on. It does not publish anything; <code>-p</code> does.	labs/04
Health check	A command Docker runs regularly to ask "is the app working?". <code>docker ps</code> shows <code>(healthy)</code> .	capstone
Image	A read-only template: a filesystem plus settings. Containers are created from images.	labs/01
Image ID	The unique ID of an image (a hash of its configuration).	labs/01
inspect	<code>docker inspect</code> : the full JSON description of a container, image, network or volume.	labs/14
Layer	One step of an image's filesystem. Layers are stacked, shared between images and cached.	labs/07
Logs	Everything the main process writes to stdout and stderr. Read with <code>docker logs</code> .	labs/02
Main process (PID 1)	The process started by <code>CMD / ENTRYPOINT</code> . When it ends, the container stops.	labs/03
Multi-stage build	A Dockerfile with several <code>FROM</code> stages; only what you <code>COPY --from</code> reaches the final image.	labs/16
Named volume	A volume you name (<code>db-data</code>); it survives until you delete it yourself.	labs/09
Network isolation	Containers on different networks cannot reach, or even resolve, each other.	labs/11
OOMKilled	"Out of memory killed": the container hit its memory limit and the kernel killed it (exit code 137).	labs/14
Port mapping (publishing)	<code>-p 8080:80</code> : traffic to port 8080 on your computer goes to port 80 in the container.	labs/04
Prune	Delete unused objects (<code>docker container prune</code> , <code>image prune</code> , <code>volume prune</code> , <code>system prune</code>). Read the warning first.	labs/18

Term	Meaning	Practise it in
Pull / push	Download an image from a registry / upload one to it.	labs/17
Read-only filesystem	<code>--read-only</code> : the container cannot change its own files; attackers cannot either.	labs/15
Registry	A server that stores images, e.g. Docker Hub, or <code>registry:3</code> running on your computer.	labs/17
Repository	A named collection of image versions in a registry, e.g. <code>sufibaba6629/docker-from-zero-web</code> .	labs/17
Restart policy	What Docker does when a container stops: <code>no</code> , <code>on-failure</code> , <code>always</code> , <code>unless-stopped</code> .	capstone
Secret	Sensitive data (a password) given as a file instead of an environment variable, so <code>docker inspect</code> does not show it.	labs/15
Service (Compose)	One entry under <code>services:</code> in a Compose file; Compose runs one or more containers for it.	labs/13
Tag	A human-friendly version label after the colon: <code>nginx:1.30-alpine</code> . Without one, Docker uses <code>latest</code> .	labs/01
tmpfs	An in-memory folder inside a container; disappears when the container stops.	labs/15
USER	Dockerfile instruction: run the container's process as this user instead of root.	labs/15
Volume	Storage managed by Docker, outside the container's own filesystem, so data survives the container.	labs/09
WORKDIR	Dockerfile instruction: the folder later instructions (and the container) work in.	labs/06
WSL2	Windows Subsystem for Linux: a real Linux on Windows; the recommended terminal for these labs on Windows.	docs/02

Interview questions

Questions you can expect in a junior DevOps or developer interview about Docker. Answer out loud first, then open the answer. Every answer points to the place in this lab where you saw it happen.

Basics

1. What is the difference between an image and a container?

▼ Answer

An image is a read-only template: a filesystem plus settings (command, environment, ports). A container is a running or stopped instance created from an image, with its own writable layer on top. One image can run as many containers. Like a class and its objects, or a recipe and the meals cooked from it. ([labs/01](#))

2. How is a container different from a virtual machine?

▼ Answer

A virtual machine runs a whole operating system with its own kernel on emulated hardware. A container is an isolated process that shares the host's kernel, so it starts in milliseconds and uses far less memory. The trade-off: weaker isolation than a VM, and Linux containers need a Linux kernel (Docker Desktop provides one in a small VM). ([docs/01](#))

3. Why does a container stop by itself?

▼ Answer

A container lives exactly as long as its main process (PID 1). When that process finishes or crashes, the container stops. `docker run ubuntu:26.04` stops immediately because `bash` has no terminal and exits. Check `docker ps -a` for the exit code and `docker logs` for the reason. ([labs/03](#), [troubleshooting/01](#))

4. What do exit codes 0, 1 and 137 tell you?

▼ Answer

0: the process finished normally. 1 (or 2, 3, ...): the application reported an error, read the logs. 137 = 128 + 9: the process was killed with SIGKILL, typically by `docker kill` or by the kernel because the container exceeded its memory limit (`docker inspect` then shows `OOMKilled: true`). ([labs/14](#))

5. What does `-p 8080:80` mean, and what does `EXPOSE` do?

▼ Answer

`-p 8080:80` publishes the container's port 80 on port 8080 of the host: host first, container second. `EXPOSE` in a Dockerfile only documents which port the application uses; it publishes nothing. ([labs/04](#))

6. A container is running and the port is published, but the browser gets no answer. What do you check?

▼ Answer

`docker ps` (is the mapping right, host:container?), `docker logs` (on which address and port did the app start?), `docker exec` to test from inside the container. A common cause: the app listens on 127.0.0.1 inside the container, which only accepts connections from inside; it must listen on 0.0.0.0. Another: the mapping points to a container port nothing listens on. ([troubleshooting/07](#))

Images and Dockerfiles

7. What is the difference between `CMD` and `ENTRYPOINT` ?

▼ Answer

`ENTRYPOINT` is the fixed program; `CMD` is the default command or the default arguments to the entrypoint. Anything you type after the image name in `docker run` replaces `CMD`, not `ENTRYPOINT`. ([labs/06](#))

8. What are layers, and why does the order of Dockerfile instructions matter?

▼ Answer

Instructions that change files (`RUN`, `COPY`, `ADD`) each create a layer. Docker caches layers and reuses them while nothing they depend on has changed; once one layer changes, all layers after it are rebuilt. So copy the dependency list and install dependencies first, and copy the frequently changing source code last. ([labs/07](#))

9. What is the build context, and what is `.dockerignore` for?

▼ Answer

The build context is the folder given to `docker build`; it is sent to the builder and `COPY` can only use files from it. `.dockerignore` excludes files from it: faster builds, smaller images, and no `.env` files, keys or `.git` folders accidentally copied into an image. ([labs/08](#))

10. How do you make an image smaller?

▼ Answer

A slim or alpine base image, a multi-stage build that leaves compilers and build tools behind, no package-manager caches (`--no-cache-dir`, `--no-cache`), only the packages you need, a `.dockerignore`, and combining related commands in one `RUN`. Measure with `docker images` and `docker history`. ([labs/16](#))

11. Why should you not use the `latest` tag in production?

▼ Answer

`latest` is just a tag name that moves whenever a new version is pushed. The same command can give different images on different days, so builds are not repeatable and rollbacks are unclear. Pin a version (`nginx:1.30-alpine`), or a digest for full certainty. ([docs/03](#))

12. What is a multi-stage build?

▼ Answer

A Dockerfile with several `FROM` stages. Early stages build or install things with all the tools they need; the final stage starts from a small base and copies only the results (`COPY --from=builder ...`). The tools never reach the final image. ([labs/16](#))

Data and networking

13. What is the difference between a volume and a bind mount?

▼ Answer

A volume is storage managed by Docker (`docker volume ls`); it is the right place for database data. A bind mount shows a specific folder of the host inside the container; it is ideal for development (edit on the host, see it live) and for config files. Both survive the container. ([labs/09](#), [labs/10](#))

14. You deleted a database container and recreated it. The data is gone. Why could that be?

▼ Answer

The data was in the container's own writable layer or in an anonymous volume, not in a named volume; or the new container mounts a different volume name (a typo, or a different Compose project name); or the volume is mounted at the wrong path (PostgreSQL 18 expects `/var/lib/postgresql`); or someone ran `docker compose down -v` . ([troubleshooting/04](#), [troubleshooting/10](#))

15. How do containers find each other?

▼ Answer

On a user-defined network (and every Compose project has one), Docker's built-in DNS resolves container names and Compose service names to the container's IP address. The default `bridge` network has no name resolution. ([labs/11](#))

16. Why not connect to other containers by IP address?

▼ Answer

Container IPs change when containers are recreated. Names stay the same, and Docker's DNS always returns the current address. ([labs/11](#))

17. How would you stop a web container from reaching the database directly?

▼ Answer

Put them on different networks: web and API on a frontend network, API and database on a backend network. The web container cannot even resolve the database's name. That is exactly what the capstone does. ([capstone](#))

Compose and operations

18. What does `docker compose up -d` create, and what does `docker compose down` remove?

▼ Answer

`up` creates a network for the project, the named volumes, and one container per service (building images if needed). `down` removes the containers and networks but keeps named volumes; `down -v` also deletes the volumes and their data. ([labs/13](#))

19. `depends_on` is set, but the API still fails because the database is not ready. Why, and what is the fix?

▼ Answer

Plain `depends_on` only waits until the database container has started, not until PostgreSQL accepts connections. Give the database a `healthcheck` and use `depends_on: db: condition: service_healthy`, as the capstone does. ([capstone](#))

20. How do you debug a container that keeps restarting?

▼ Answer

`docker ps -a` for the status and exit code, `docker logs` (with `--tail`) for the error message, `docker inspect` for the configuration (environment, mounts, command, `oomkilled`), and if it stays up long enough, `docker exec` to look inside. Read the evidence before changing anything. ([troubleshooting/](#))

21. How do you limit a container's memory and CPU, and how do you see its usage?

▼ Answer

`docker run --memory=256m --cpus=0.5 ...` (in Compose: `deploy.resources.limits`). `docker stats` shows live CPU, memory, network and disk I/O; `docker top` shows the processes. ([labs/14](#))

Security

22. Why run containers as a non-root user?

▼ Answer

If the application is compromised, the attacker gets the rights of its user. As root inside the container they can change any file in it, and a container escape bug becomes much more dangerous. Add a user in the Dockerfile and switch to it with `USER`. ([labs/15](#))

23. Why are environment variables not ideal for passwords?

▼ Answer

They are visible to anyone who can run `docker inspect`, they appear in process listings and crash reports, and they are inherited by child processes. Pass secrets as files (Compose `secrets:`, `*_FILE` variables) or use a secrets manager. ([labs/15](#))

24. Name five basic ways to make a container more secure.

▼ Answer

Official or trusted, version-pinned base images; a minimal image; a non-root `USER` ; `--read-only` with a `tmpfs` for temporary files; `--cap-drop ALL` ; secrets as files; only the ports you need published; resource limits; scanning images for known vulnerabilities. ([docs/19](#))

25. A colleague wants to free disk space with `docker system prune -a --volumes` . What do you tell them?

▼ **Answer**

That it deletes every stopped container, every unused network, every image not used by a container (so everything has to be downloaded or rebuilt again), the build cache, and every volume not used by a container, including database data of stopped projects. Check `docker system df` first, and prefer targeted commands with filters. ([labs/18](#))

Knowledge checklist

Tick a line only when you have **done** it yourself, not just read about it. Each line links to where you practise it.

- ☐ I understand what Docker is · [docs/01](#)
- ☐ I understand images · [docs/03](#), [labs/01](#)
- ☐ I understand containers · [docs/04](#), [labs/03](#)
- ☐ I can run a container · [labs/01](#)
- ☐ I can inspect a container · [labs/02](#), [labs/14](#)
- ☐ I can read logs · [labs/02](#), [docs/16](#)
- ☐ I can execute commands inside containers · [labs/02](#)
- ☐ I understand port mapping · [labs/04](#)
- ☐ I can write a Dockerfile · [labs/06](#)
- ☐ I can build an image · [labs/06](#)
- ☐ I understand image layers · [labs/07](#)
- ☐ I understand build cache · [labs/07](#)
- ☐ I understand .dockerignore · [labs/08](#)
- ☐ I understand volumes · [labs/09](#)
- ☐ I understand bind mounts · [labs/10](#)
- ☐ I understand networking · [labs/11](#)
- ☐ I can connect containers · [labs/12](#)
- ☐ I can use Docker Compose · [labs/13](#)
- ☐ I can troubleshoot containers · [troubleshooting/](#)
- ☐ I understand basic Docker security · [labs/15](#)
- ☐ I can optimize an image · [labs/16](#)
- ☐ I can build a multi-container application · [capstone/](#)

Extra skills you practised

- ☐ I can limit memory and CPU, and recognise an out-of-memory kill (exit code 137) · [labs/14](#)
- ☐ I can push an image to a registry and pull it somewhere else · [labs/17](#)
- ☐ I can free disk space without deleting data I need · [labs/18](#)
- ☐ I follow Observe → Investigate → Root cause → Fix → Verify instead of guessing · [troubleshooting/](#)